



# 东邪的机器学习之旅

作者: dx

时间: 2025, 8, 19

版本: 1.0

自定义: 信息



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

# 目录

|                                             |           |
|---------------------------------------------|-----------|
| <b>第 1 章 PCA</b>                            | <b>1</b>  |
| 1.1 协方差矩阵理解                                 | 1         |
| 1.2 SVD 奇异值分解                               | 2         |
| 1.2.1 为什么要把矩阵转化为向量乘积                        | 3         |
| 1.2.2 为什么还需要 SVD 分解                         | 3         |
| 1.2.3 如何寻找正交化基底                             | 4         |
| 1.2.4 奇异值分解计算方法                             | 5         |
| 1.2.5 奇异值分解也可以满足不满秩的情况                      | 5         |
| 1.2.6 为什么可以去掉小的 $\sigma$ 项                  | 6         |
| <b>第 2 章 QR 分解</b>                          | <b>9</b>  |
| 2.1 转置子空间等知识回顾                              | 10        |
| 2.2 最小二乘法                                   | 12        |
| 2.3 QR 分解操作                                 | 15        |
| 2.4 QR 分解迭代法-数值分析                           | 17        |
| <b>第 3 章 LLL 格基</b>                         | <b>20</b> |
| 3.1 LLL 最短向量的意义                             | 23        |
| 3.2 LLL 的时间复杂度                              | 26        |
| <b>第 4 章 基于树的方法</b>                         | <b>28</b> |
| 4.1 回归树                                     | 28        |
| 4.1.1 回归树的具体计算                              | 29        |
| 4.2 装代法                                     | 31        |
| 4.3 随机森林                                    | 32        |
| <b>第 5 章 基础概念</b>                           | <b>33</b> |
| <b>第 6 章 《Attention Is All You Need》中文版</b> | <b>35</b> |
| 6.1 摘要                                      | 35        |
| 6.2 引言                                      | 35        |
| 6.3 背景                                      | 35        |
| 6.4 模型架构                                    | 36        |
| 6.4.1 编码器与解码器堆栈                             | 36        |
| 6.4.2 注意力                                   | 36        |
| 6.4.2.1 缩放点积注意力                             | 36        |
| 6.4.2.2 多头注意力                               | 37        |
| 6.4.2.3 注意力在模型中的应用                          | 37        |
| 6.4.3 逐位置前馈网络                               | 37        |
| 6.4.4 嵌入与 Softmax                           | 37        |
| 6.4.5 位置编码                                  | 37        |
| 6.5 为什么使用自注意力                               | 38        |
| 6.6 训练                                      | 38        |

|              |                                                              |           |
|--------------|--------------------------------------------------------------|-----------|
| 6.6.1        | 训练数据与批处理                                                     | 38        |
| 6.6.2        | 硬件与训练计划                                                      | 38        |
| 6.6.3        | 优化器                                                          | 39        |
| 6.6.4        | 正则化                                                          | 39        |
| 6.7          | 实验结果                                                         | 39        |
| 6.7.1        | 机器翻译                                                         | 39        |
| 6.7.2        | 模型变体                                                         | 39        |
| 6.7.3        | 英语成分句法分析                                                     | 40        |
| 6.8          | 结论                                                           | 40        |
| <b>第 7 章</b> | <b>Attention Is All You Need</b>                             | <b>41</b> |
| 7.1          | Abstract                                                     | 41        |
| 7.2          | Introduction                                                 | 41        |
| 7.3          | Background                                                   | 41        |
| 7.4          | Model Architecture                                           | 42        |
| 7.4.1        | Encoder and Decoder Stacks                                   | 42        |
| 7.4.2        | Attention                                                    | 42        |
| 7.4.2.1      | Scaled Dot-Product Attention                                 | 43        |
| 7.4.2.2      | Multi-Head Attention                                         | 43        |
| 7.4.2.3      | Applications of Attention in our Model                       | 43        |
| 7.4.3        | Position-wise Feed-Forward Networks                          | 44        |
| 7.4.4        | Embeddings and Softmax                                       | 44        |
| 7.4.5        | Positional Encoding                                          | 44        |
| 7.5          | Why Self-Attention                                           | 44        |
| 7.6          | Training                                                     | 45        |
| 7.6.1        | Training Data and Batching                                   | 45        |
| 7.6.2        | Hardware and Schedule                                        | 45        |
| 7.6.3        | Optimizer                                                    | 46        |
| 7.6.4        | Regularization                                               | 46        |
| 7.7          | Results                                                      | 46        |
| 7.7.1        | Machine Translation                                          | 46        |
| 7.7.2        | Model Variations                                             | 46        |
| 7.7.3        | English Constituency Parsing                                 | 47        |
| 7.8          | Conclusion                                                   | 47        |
| <b>第 8 章</b> | <b>《Deep Neural Networks for YouTube Recommendations》中文版</b> | <b>48</b> |
| 8.1          | 摘要                                                           | 48        |
| 8.2          | 引言                                                           | 48        |
| 8.3          | 系统概述                                                         | 48        |
| 8.4          | 候选生成                                                         | 49        |
| 8.4.1        | 将推荐建模为分类                                                     | 49        |
| 8.4.2        | 模型架构                                                         | 49        |
| 8.4.3        | 异质信号                                                         | 50        |
| 8.4.4        | 标签与上下文选择                                                     | 50        |
| 8.4.5        | 特征与深度实验                                                      | 50        |

---

|              |                                                         |           |
|--------------|---------------------------------------------------------|-----------|
| 8.5          | 排序 . . . . .                                            | 51        |
| 8.5.1        | 特征表示 . . . . .                                          | 51        |
| 8.5.2        | 期望观看时长建模 . . . . .                                      | 52        |
| 8.5.3        | 隐藏层实验 . . . . .                                         | 52        |
| 8.6          | 结论 . . . . .                                            | 52        |
| <b>第 9 章</b> | <b>Deep Neural Networks for YouTube Recommendations</b> | <b>54</b> |
| 9.1          | Abstract . . . . .                                      | 54        |
| 9.2          | Introduction . . . . .                                  | 54        |
| 9.3          | System Overview . . . . .                               | 55        |
| 9.4          | Candidate Generation . . . . .                          | 55        |
| 9.4.1        | Recommendation as Classification . . . . .              | 55        |
| 9.4.2        | Model Architecture . . . . .                            | 56        |
| 9.4.3        | Heterogeneous Signals . . . . .                         | 56        |
| 9.4.4        | Label and Context Selection . . . . .                   | 57        |
| 9.4.5        | Experiments with Features and Depth . . . . .           | 57        |
| 9.5          | Ranking . . . . .                                       | 58        |
| 9.5.1        | Feature Representation . . . . .                        | 58        |
| 9.5.2        | Modeling Expected Watch Time . . . . .                  | 59        |
| 9.5.3        | Experiments with Hidden Layers . . . . .                | 60        |
| 9.6          | Conclusions . . . . .                                   | 60        |

# 第 1 章 PCA

## 定义 1.1 (Principal Component Analysis)

主成分分析 (PCA) 是一种无监督学习算法，用于降维并识别数据中最具信息量的特征。

它的本质思想是：假设数据关系具有线性结构，并将其投影到一个由正交向量组成的子空间中，使得方差最大化。这样的向量称为主成分，它们决定了数据变化最大（信息量最高）的方向。

另一种等价的解释是：PCA 可以看作一种线性投影，它最小化了原始点与其投影点之间的均方根 (RMS) 距离。

### PCA 的工作原理

首先，特征矩阵需要进行中心化处理，以确保第一主成分对应的是数据的最大变化方向，而不仅仅是平均值。通常，求主成分的方法有两种主要思路：第一，协方差矩阵的特征值与特征向量分解。协方差矩阵反映了不同变量之间的线性相关程度。该矩阵的特征向量给出了数据方差最大的方向，而特征值则表示该方向上的方差大小。与某个特征向量对应的特征值刻画了该向量对解释数据方差的贡献。特征值越大，该主成分的重要性就越高。通常，只选择能够解释预设方差比例（如 95%）的主成分。

## 1.1 协方差矩阵理解

### 定义 1.2 (3.3.1)

设  $(X_1, X_2, \dots, X_n)$  是  $n$  维随机变量，若  $E(X_1), E(X_2), \dots, E(X_n)$  存在，则称

$$(E(X_1), E(X_2), \dots, E(X_n))$$

为  $(X_1, X_2, \dots, X_n)$  的数学期望，简称期望。

固定  $i, j$ ，若

$$E([X_i - E(X_i)][X_j - E(X_j)])$$

存在，则称其为  $X_i$  与  $X_j$  的协方差 (covariance)，记作  $\text{Cov}(X_i, X_j)$ ，即

$$\text{Cov}(X_i, X_j) = E([X_i - E(X_i)][X_j - E(X_j)]).$$

易知， $E(X_i)$  和  $E(X_j)$  存在是  $\text{Cov}(X_i, X_j)$  存在的必要条件。

若对  $i, j = 1, 2, \dots, n$ ，所有  $\text{Cov}(X_i, X_j)$  均存在，则称  $n$  阶方阵

$$\Sigma = (b_{ij})_{n \times n}, \quad b_{ij} = \text{Cov}(X_i, X_j)$$

为  $(X_1, X_2, \dots, X_n)$  的协方差矩阵 (covariance matrix)。

### 东邪的 tip 1.1 (方差与协方差总结)

#### 1. 定义

$$\text{Var}(X) = E[(X - E[X])^2], \quad \text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

#### 2. 数学关系

$$\text{Var}(X) = \text{Cov}(X, X)$$

方差是协方差的特例。

#### 3. 性质比较

| 性质   | 方差 $\text{Var}(X)$             | 协方差 $\text{Cov}(X,Y)$                 |
|------|--------------------------------|---------------------------------------|
| 定义   | 单变量的离散程度                       | 两变量的联动程度                              |
| 符号   | $\geq 0$                       | 可正、可负、或为 0                            |
| 几何意义 | 数据点围绕均值的分布宽度                   | 点云的倾斜方向、线性关系                          |
| 特殊情况 | $\text{Var}(X) = 0 \iff X$ 为常数 | $\text{Cov}(X,Y) = 0 \nRightarrow$ 独立 |

里面的  $E(\cdot)$  就是期望算子，它的作用就是“取加权平均”。情景：两门考试成绩。设三名学生的语文与数学成绩分别为 A:  $X = 50, Y = 55$ , B:  $X = 60, Y = 65$ , C:  $X = 70, Y = 75$ 。

#### 1. 算平均值（期望）：

$$E[X] = \frac{50 + 60 + 70}{3} = 60, \quad E[Y] = \frac{55 + 65 + 75}{3} = 65.$$

#### 2. 偏离均值：

$$\text{A: } X - 60 = -10, Y - 65 = -10;$$

$$\text{B: } X - 60 = 0, Y - 65 = 0;$$

$$\text{C: } X - 60 = 10, Y - 65 = 10.$$

#### 3. 偏离乘积：

$$\text{A: } (-10)(-10) = 100, \quad \text{B: } 0 \cdot 0 = 0, \quad \text{C: } 10 \cdot 10 = 100.$$

#### 4. 取平均（期望）得到协方差：

$$\text{Cov}(X,Y) = E[(X - E[X])(Y - E[Y])] = \frac{100 + 0 + 100}{3} = \frac{200}{3} \approx 66.7.$$

结果解读：协方差为正，说明语文高的一般数学也高（正相关）；若为负则一高一低（负相关）；若接近 0，则两者几乎无线性关系。协方差矩阵就是把所有不同的  $X_i$  和  $Y_j$  的关联性在一个矩阵中记录出来。

#### 东邪的 tip 1.2

进一步来说减法是把所有的  $x$  轴上的点左移或者右移， $Y$  轴的点上移或者下移，两个合起来的操作就是把坐标系切换成了以他们均值为原点的坐标系。他们的乘积就是每一个坐标和原点围成矩形的面积，加权的方式可以让你觉得哪一部分的重要凸显，比如说我希望进缩小极大偏离平均值的那项对整体的影响我就可以给他加一个小的权重让他生成的矩阵面积缩小。

- 协方差不仅是“面积”，还是一种趋势的代数平均。
- 如果绝大多数点落在“同向象限”（右上、左下），那么平均面积大于 0  $\rightarrow$  正相关。
- 如果大多数点落在“反向象限”（右下、左上），平均面积小于 0  $\rightarrow$  负相关。
- 如果点云分布比较均匀，正负面积互相抵消  $\rightarrow$  协方差接近 0。

协方差的几何意义有助于理解其方向与强度，可参见下列视频：<https://www.bilibili.com/video/BV1UWYfz2EKZ>

## 1.2 SVD 奇异值分解

第二，数据矩阵的奇异值分解 (SVD)。奇异值分解将任意矩阵表示为三个矩阵的乘积：左奇异矩阵  $U$ 、奇异值对角矩阵  $S$ 、以及右奇异矩阵  $V$ 。其中，奇异值是协方差矩阵特征值的平方根（这就是为什么需要先对数据做中心化），右奇异矩阵  $V$  对应于协方差矩阵的特征向量，而左奇异矩阵  $U$  则是原始数据在主成分上的投影。因此，SVD 也可以提取出主成分，而且无需显式计算协方差矩阵。与特征分解方法相比，这种方式更高效、更数值稳定，尤其是在特征间存在强相关性、导致协方差矩阵条件数较差的情况下。特别地，scikit-learn 的 PCA 实现采用的就是这种方法，不过还结合了一些额外的技巧。

**东邪的 tip 1.3 (特征值与奇异值)**

奇异值与特征值在定义和几何意义上既有联系又有区别。对于方阵  $A \in \mathbb{R}^{n \times n}$ , 若存在  $\lambda$  使得  $Av = \lambda v$  ( $v \neq 0$ ), 则  $\lambda$  称为  $A$  的特征值; 而对于任意矩阵  $A \in \mathbb{R}^{m \times n}$ , 若存在  $\sigma$  使得  $A^T A v = \sigma^2 v$  ( $v \neq 0$ ), 则  $\sigma$  称为  $A$  的奇异值, 即奇异值是  $A^T A$  的特征值的非负平方根。因此奇异值永远非负, 而特征值可能为正或负。进一步地, 如果  $A$  是对称矩阵, 则奇异值等于特征值的绝对值。例如  $A = \begin{bmatrix} -3 & 0 \\ 0 & 2 \end{bmatrix}$  的特征值为  $-3, 2$ , 而奇异值为  $3, 2$ 。

在几何意义上, 特征值描述了矩阵在某些特定方向上对向量的缩放因子, 而奇异值则表示矩阵将单位球拉伸为椭球时的半轴长度, 刻画的是整体的伸缩尺度。换句话说, 奇异值等于  $A^T A$  的特征值平方根, 而在对称矩阵的情形下, 奇异值就是特征值的绝对值。

**1.2.1 为什么要把矩阵转化为向量乘积**

一个  $m \times n$  的矩阵一共有  $mn$  个单元, 但是一列加一行只有  $m+n$  个分量。为了快速处理矩阵, 往往需要将其转化为分量乘积的形式。简单来说, 一个图像可以视为一个大型矩阵。要完美复制它, 我们需要  $8mn$  比特的信息。

高分辨率电视通常有  $m = 1080$  与  $n = 1920$ , 每秒通常有 24 帧, 而且你可能喜欢看彩色画面 (3 个颜色通道)。因此总共需要传输

$$3 \times 8 \times (1080 \times 1920 \times 24) = 3 \times 8 \times 48,470,400$$

比特每秒。这太昂贵而且不可行, 传输器也跟不上表演的速度。

当压缩做得很好时, 你几乎看不出与原始图像的差别。图像的边缘 (即灰度的突然改变) 是最难压缩的部分。如果每个  $a_{ij}$  都是无关的随机数字, 那么压缩根本不可能成功。我们完全依赖这样的事实: 邻近像素通常具有相似的灰度值。当你跨越边缘时, 会产生一个突跳。现实世界的图像中边缘无处不在, 所以动画相比静止图像更容易压缩。

**1.2.2 为什么还需要 SVD 分解**

Singular Value Decomposition,

**例题 1.12** “王牌旗”法国国旗  $A$ , 意大利国旗  $A$ , 德国国旗  $A^T$

$$A = \begin{bmatrix} a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \\ a & a & c & c & e & e \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} a & a & c & c & e & e \end{bmatrix}$$

国旗有 3 个颜色但是秩还是 1。假设这 36 个单元保持秩 1 的模式  $A = u_1 v_1^T$ , 它们甚至可以全部不相同。但是当秩变成  $r = 2$  时, 我们需要  $u_1 v_1^T + u_2 v_2^T$ 。秩是几就需要几个非线性相关的矩阵, 每个矩阵都可以写成行乘列

**例题 1.23 嵌入方形**

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

$A$  的 1 与 0 可以是 1 的方块与 0 的方块。秩仍然是 2, 我们仍然只需要两个项  $u_1 v_1^T + u_2 v_2^T$ 。一个  $6 \times 6$  图像可以由 24 个数字压缩而来; 一个  $N \times N$  图像 ( $N^2$  个数字) 可以压缩成 4 个向量  $u_1, u_2, v_1, v_2$  的  $4N$  个数字。

注意：这并不是来自 SVD 的选择！例如  $u_1 = (1, 1), u_2 = (1, 0)$  没有正交； $u_1 = (1, 1)$  与  $v_2 = (0, 1)$  也没有正交。SVD 会保证按照顺序选择秩 1 片段。

#### 东邪的 tip 1.4 (正交化的好处)

唯一性：正交基保证分解几乎唯一，不会乱选。

简洁几何意义：任何矩阵 = 旋转 + 拉伸 + 再旋转。

数值稳定：正交基避免冗余，误差不放大。

最优近似：保留前几个奇异值就得到最小误差的低秩近似。

### 1.2.3 如何寻找正交化基底

ATA 是半正定而且对称的他天然自带一组正交基，AAT 同理我们这样就得倒了两组正交基。

来自 SVD 的选择

$$AA^T u_i = \sigma_i^2 u_i, \quad A^T A v_i = \sigma_i^2 v_i, \quad A v_i = \sigma_i u_i$$

$$AA^T A v_i = \sigma_i^2 A v_i \Rightarrow A v_i \text{ 是 } AA^T \text{ 的特征向量, 对应特征值 } \sigma_i^2.$$

因此  $A v_i = u_i$ ，但此时的  $u_i$  还不是单位向量，需要计算  $A v_i$  的长度：

$$\|A v_i\|^2 = (A v_i)^T (A v_i) = v_i^T (A^T A) v_i = v_i^T (\sigma_i^2 v_i) = \sigma_i^2 \|v_i\|^2 = \sigma_i^2.$$

$$\Rightarrow \|A v_i\| = \sigma_i, \quad u_i = \frac{1}{\sigma_i} A v_i.$$

#### 东邪的 tip 1.5

因为特征向量乘一个系数还是特征向量所以要看清到底指的那个，比如这里要求的就是单位化的。单位化的一大好处就是满秩正交矩阵乘自己的转置等于单位矩阵可以消除项。

由  $A v_i = \sigma_i u_i$  可得

$$A v_i v_i^T = (A v_i) v_i^T = \sigma_i u_i v_i^T,$$

它表示  $A$  在第  $i$  个方向上的秩一部分：该矩阵将任意输入向量在  $v_i$  方向上的分量提取出来，并映射到  $u_i$  方向上，因而秩为 1（若  $\sigma_i = 0$  则为 0）。

整体的式子如下

$$A V V^T = U \Sigma V^T, \quad V V^T = I, \quad \Rightarrow A = U \Sigma V^T.$$

其中  $\Sigma$  是一个对角矩阵

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r).$$

$$A = \begin{bmatrix} u_1 & u_2 \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix} \begin{bmatrix} v_1^T \\ v_2^T \end{bmatrix} \quad \text{或更简单的} \quad A \begin{bmatrix} v_1 & v_2 \end{bmatrix} = \begin{bmatrix} \sigma_1 u_1 & \sigma_2 u_2 \end{bmatrix}.$$

$\sigma$  矩阵在中间，是因为矩阵运算中是后一个矩阵对前一个矩阵作用产生变换，这里  $\sigma$  是为了得到  $u$  的系数（即缩放），所以要放在  $U$  矩阵的前面。

### 1.2.4 奇异值分解计算方法

SVD 的  $u$ 's 称为左奇异向量 ( $AA^T$  的单位特征向量),  $v$ 's 称为右奇异向量 ( $A^T A$  的单位特征向量),  $\sigma$ 's 是奇异值, 就是  $AA^T$  与  $A^T A$  的相同特征值的平方根:

在范例 3 (嵌入方形) 中, 下列是对称矩阵  $AA^T$  与  $A^T A$ :

$$AA^T = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad A^T A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}.$$

它们的行列式是 1, 所以  $\lambda_1 \lambda_2 = 1$ , 迹 (对角线的和) 是 3:

$$\det \begin{bmatrix} 1 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = \lambda^2 - 3\lambda + 1 = 0,$$

得到

$$\lambda_1 = \frac{3+\sqrt{5}}{2}, \quad \lambda_2 = \frac{3-\sqrt{5}}{2}.$$

$\lambda_1, \lambda_2$  的平方根是

$$\sigma_1 = \frac{\sqrt{5}+1}{2}, \quad \sigma_2 = \frac{\sqrt{5}-1}{2},$$

且  $\sigma_1 \sigma_2 = 1$ 。

—

最接近  $A$  的秩 1 矩阵是  $\sigma_1 u_1 v_1^T$ , 误差只有  $\sigma_2 \approx 0.6$ , 是最佳可能。

$AA^T$  与  $A^T A$  的正交单位特征向量是:

$$u_1 = \begin{bmatrix} 1 \\ \sigma_1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} \sigma_1 \\ -1 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ -\sigma_1 \end{bmatrix},$$

全部除以  $\sqrt{1 + \sigma_1^2}$ 。

像通常有满秩, 但是他们确实有低效率的 (low effective) 秩, 这就表示: 很多奇异值很小, 可以被设定成零。我们传输的是低秩的近似压缩时我们丢弃小的  $\sigma$  不会严重损失图像的品质,

#### 东邪的 tip 1.6 (对角化的好处)

1. 运算简化:  $A^k = P D^k P^{-1}$ , 复杂矩阵运算化为标量运算。
2. 结构清晰: 在特征向量基下, 矩阵只表现为伸缩 (按特征值缩放)。
3. 理论方便: 如解微分方程、研究马尔可夫链, 化为一维问题。
4. 易于分析: 长期行为由最大特征值主导, 可用于近似与压缩。

奇异值分解是线性代数的一个亮点,  $A$  的任意  $m \times n$  矩阵, 方形或矩形, 它的秩是  $r$ 。我们会对角化  $A$ , 但不是用  $X^{-1} A X$ ,  $X$  里面的特征向量有三个大问题: 他们通常不是正交, 不一定有足够的特征向量, 而且  $Ax = x$  要求  $A$  是方形。A 的奇异向量以完美的方式解决了全部的问题。

### 1.2.5 奇异值分解也可以满足不满秩的情况

(使用向量  $u$ 's 与  $v$ 's 产生 4 个基础子空间的基底:)

- |                       |                          |
|-----------------------|--------------------------|
| $u_1, \dots, u_r$     | 是列空间的一组正交单位基底            |
| $u_{r+1}, \dots, u_m$ | 是左零空间 $N(A^T)$ 的一组正交单位基底 |
| $v_1, \dots, v_r$     | 是行空间的一组正交单位基底            |
| $v_{r+1}, \dots, v_n$ | 是零空间 $N(A)$ 的一组正交单位基底    |

这些  $v'$  s 与  $u'$  s 负责  $A$  的行空间与列空间, 我们还有  $n-r$  个  $v'$  s 与  $m-r$  个  $u'$  s, 分别来自零空间  $N(A)$  与左零空间  $N(A^T)$ 。他们自动与前面  $v'$  s 与  $u'$  s 正交 (因为整个零空间是正交)。我们现在引入  $V$  与  $U$  的所有  $v'$  s 与  $u'$  s, 所以矩阵变成方形, 我们仍然有  $AV = U\Sigma$ 。

$$(m \times n)(n \times n) \quad AV = U\Sigma \quad (m \times m)(m \times n)$$

$$A \begin{bmatrix} v_1 & \cdots & v_r & \cdots & v_n \end{bmatrix} = \begin{bmatrix} u_1 & \cdots & u_r & \cdots & u_n \end{bmatrix} \begin{bmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & \sigma_r \end{bmatrix} \quad (3)$$

新的  $\Sigma$  是  $m \times n$ , 它就是方程式 (2) 的  $r \times r$  矩阵加上  $m-r$  个额外零行以及  $n-r$  个零列, 真正的改变在于  $U$  与  $V$  的形状。这些是方形矩阵且  $V^{-1} = V^T$ , 所以  $AV = U\Sigma$  变成  $A = U\Sigma V^T$ , 这就是奇异值分解。

我可以把  $V^T$  的行乘来自  $U\Sigma$  的列  $u_i\sigma_i$ :

$$\text{SVD: } A = U\Sigma V^T = \sigma_1 u_1 v_1^T + \cdots + \sigma_r u_r v_r^T. \quad (4)$$

然后我们吧  $\sigma$  以及后面按降序排列最大的排前面。

**范例 1.** 什么时候

$$A = U\Sigma V^T \quad (\text{奇异值分解})$$

与

$$A = X\Lambda X^{-1} \quad (\text{特征值分解})$$

相等?

**解:** 矩阵  $A$  需要具有正交特征向量, 才能使得  $X = U = V$ 。如果  $\Lambda = \Sigma$ , 则  $A$  还需要满足特征值  $\lambda \geq 0$ 。因此  $A$  必须是一个正半定 (或正定) 对称矩阵。

只有在这种情况下:

$$A = X\Lambda X^{-1} = Q\Lambda Q^T$$

才能与

$$A = U\Sigma V^T$$

一致。

## 1.2.6 为什么可以去掉小的 $\sigma$ 项

**例:** 矩阵  $A$  的奇异值稳定性

原矩阵

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

—

**1. 未去掉最后一行时:**

$$A^T A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 9 \end{bmatrix}, \quad \Rightarrow \sigma = 3, 2, 1.$$

2. 去掉最后一行后：得到  $3 \times 3$  矩阵

$$A' = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}, \quad A'^T A' = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 4 \end{bmatrix},$$

其特征值为 1, 4, 9, 对应奇异值仍为

$$\sigma = 3, 2, 1.$$

**3. 结论：**去掉  $A$  的零行（或零列）不会改变非零奇异值。原因是：零行/零列只会在  $A^T A$  或  $AA^T$  中产生额外的 0 特征值，因此不会影响已有的非零奇异值。

### 命题 1.1 (SVD 的几何意义)

设矩阵  $A \in \mathbb{R}^{m \times n}$  的奇异值分解为

$$A = U \Sigma V^T,$$

则有以下几何解释：

1.  $A$  可以分解为 旋转-拉伸-旋转的组合；
2.  $A$  将单位圆上的向量  $x$  变换到椭圆上的  $Ax$ ；
3.  $A$  的算子范数为

$$\|A\| = \sigma_1,$$

其中  $\sigma_1$  是最大的奇异值，表示最大的增长因子

$$\max_{x \neq 0} \frac{\|Ax\|}{\|x\|};$$

4.  $A$  的极分解为

$$A = QS, \quad Q = UV^T, \quad S = V \Sigma V^T;$$

5.  $A$  的伪逆为

$$A^+ = V \Sigma^+ U^T,$$

它可将列空间中的  $Ax$  映射回行空间中的  $x$ 。

### 定理 1.1 (基于 SVD 的 PCA 构造过程如下：)

1. 首先进行数据中心化，如果没有指定主成分的数量，则主成分数至少在样本数和特征数之间确定；
2. 然后对中心化后的数据矩阵应用奇异值分解 (SVD)；
3. 将 `svd_flip_vector` 方法应用于矩阵  $U$ ，该方法在矩阵  $U$  的每一列中找到最大模元素，提取其符号，并用这些符号乘以矩阵  $U$ ，以保证输出结果的确定性；
4. 每个主成分的解释方差计算为相应奇异值平方除以  $n_{\text{samples}} - 1$ 。然后根据下式计算变换后的数据，主成分的数量取决于所需的保留数量：

$$X_{\text{new}} = X \cdot V = U \cdot S \cdot V^T \cdot V = U \cdot S$$

### 定理 1.2 (PCA 的附加特性)

(1) 解释方差系数：每个主成分的解释方差系数可以通过变量 `explained_variance_ratio_` 获得，表示数据集在每个主成分轴上的方差比例。

(2) 数据恢复：使用 `inverse_transform()` 方法进行数据恢复，其本质是对 PCA 投影进行逆变换：

$$X_{\text{recovered}} = X_{d-\text{proj}} W_d^T,$$

其中  $W_d^T$  是前  $d$  个主成分所组成的矩阵。数据的恢复会存在与丢弃的方差量成比例的损失；原始数据与恢复数据之间的距离平方的平均值称为 重构误差。

(3) 增量 PCA：实现为 `IncrementalPCA` 类，允许在处理大规模数据集时更加高效，可以将数据拆分成小块并逐步存储在内存中进行训练。

(4) 随机 PCA：通过设置参数 `svd_solver='randomized'` 来实现，使用随机算法快速计算近似的  $d$  个主成分。该方法基于随机投影的假设，即数据在投影到低维子空间时仍能较好地保持其结构和性质。但方法的精确度可能较低。

(5) 核 PCA：实现为 `KernelPCA` 类，允许使用核函数进行复杂的非线性投影。类似于支持向量机 (SVM)，其本质是将原始空间中的非线性问题转化为高维特征空间中的线性边界。



### 定理 1.3 (PCA 的替代方法)

尽管主成分分析 (PCA) 是最常用的降维算法之一，但在一些情况下，根据数据类型不同，也存在一些更为合适的替代方法：

**1.2.6.0.1 LLE (Locally Linear Embedding)** 局部线性嵌入是一种通过邻居点的线性组合来表示每个数据点，并在低维空间中恢复这些组合的算法。这样可以保持数据的非线性几何结构，在某些对全局性质要求不高的任务中十分有用。然而，该方法计算复杂度较高，并且可能对噪声敏感。

**1.2.6.0.2 t-SNE (t-Distributed Stochastic Neighbor Embedding)** t-SNE 是一种将高维空间中点之间的相似性转化为概率的算法，并试图最小化高维与低维空间中概率分布的差异。t-SNE 在可视化高维数据时非常有效，但它可能会扭曲数据的整体结构，因为它只考虑点在原始空间中的局部邻近性，而忽略了线性依赖关系。

**1.2.6.0.3 UMAP (Uniform Manifold Approximation and Projection)** 统一流形近似与投影是一种适用于数据可视化的算法，其思想是数据位于某个可以用邻居图近似的同质流形上。该方法考虑了数据的整体结构，相比 t-SNE 更能适应不同类型的数据，并且在处理噪声与异常值时表现更好。

**1.2.6.0.4 Autoencoders** 自编码器是一类基于神经网络的模型，通过训练编码器将输入数据转换为低维表示，再通过解码器从该表示中恢复原始数据。自编码器不仅可以用于数据压缩与降维，还可用于去噪和特征提取等多种任务。



## 第2章 QR 分解

### 定义 2.1 (正交单位向量组)

向量组  $\mathbf{q}_1, \dots, \mathbf{q}_n$  是正交单位, 若

$$\mathbf{q}_i^T \mathbf{q}_j = \begin{cases} 0, & i \neq j \text{ (正交向量)}, \\ 1, & i = j \text{ (单位向量: } \|\mathbf{q}_i\| = 1) \end{cases}.$$

一个有正交列的矩阵都将被记为特征字母  $Q$ 。



矩阵  $Q$  很容易处理, 因为

$$Q^T Q = I.$$

在矩阵语言中再一次说明: 若列向量  $\mathbf{q}_1, \dots, \mathbf{q}_n$  正交单位, 则  $Q$  的列正交单位; 注意  $Q$  不一定需要是方阵 (列正交即可, 称为半正交矩阵)

矩阵  $Q$  是方阵时, 若满足

$$Q^T Q = I,$$

则有  $Q^T = Q^{-1}$ , 也就是说转置矩阵等于逆矩阵。

进一步说明: 当  $Q$  是矩形矩阵时, 只要  $Q^T Q = I$ , 此时  $Q^T$  只是  $Q$  的左逆矩阵。若  $Q$  是方阵, 我们还可以得到

$$Q Q^T = I,$$

因此  $Q^T$  既是左逆也是右逆, 即为  $Q$  的双边逆矩阵。在这种情形下,  $Q$  的行向量也像列向量一样正交归一, 我们称这样的方阵  $Q$  为正交矩阵。

### 东邪的 tip 2.1 (正交化的好处需要牢记)

只有弄清楚了我们分解的目的才能聚焦在分解的手段

一个置换矩阵是一个方阵, 它的每一行、每一列里都有且仅有一个元素是 1, 其余元素全是 0。

换句话说, 置换矩阵是单位矩阵  $I$  的行或列经过某种重新排列 (permutation) 得到的矩阵。置换矩阵都是正交矩阵

**例题 2.1 反射** 若  $u$  是任意的单位向量, 令

$$Q = I - 2uu^T.$$

注意  $uu^T$  是一个矩阵, 而  $u^T u$  是一个数字。由于  $\|u\|^2 = 1$ , 可得

$$Q^T = I - 2uu^T = Q, \quad Q^T Q = I - 4uu^T + 4uu^T uu^T = I. \quad (2)$$

因此, 反射矩阵  $I - 2uu^T$  对称而且正交。如果把它平方, 你会得到单位矩阵:

$$Q^2 = Q^T Q = I.$$

这意味着通过镜子反射两次会回到原始状态, 就好像  $(-1)^2 = 1$  一样。注意在式 (2) 的  $4uu^T uu^T$  中, 利用了  $u^T u = 1$ 。

### 东邪的 tip 2.2

正交矩阵还能避免求逆矩阵的麻烦。

旋转会保有每个向量的长度，反射 (reflections) 也是，置换 (permutations) 也是，用任何正交矩阵去乘也一样——长度与角度没有改变。

证明：

$$\|Qx\|^2 = \|x\|^2,$$

因为

$$(Qx)^T(Qx) = x^T Q^T Q x = x^T I x = x^T x.$$

### 定理 2.1 (正交矩阵保持长度与内积)

若  $Q$  有正交单位列 (即  $Q^T Q = I$ )，它会保持向量的长度不变：

$$\text{对于每个向量 } x, \quad \|Qx\| = \|x\|. \quad (3)$$

同时， $Q$  也会保持点积：

$$(Qx)^T(Qy) = x^T Q^T Q y = x^T y.$$

这应用了  $Q^T Q = I$ ！



## 2.1 转置子空间等知识回顾

### 定理 2.2 (微积分与转置)

让我插入一点点微积分，这是极其重要的，否则我不会在讲线性代数时停下来。(实际上，这里是针对函数  $x(t)$  的线性代数。) 矩阵在这里变成了一个导数运算： $A = \frac{d}{dt}$ 。要找到这个特殊  $A$  的转置 (更准确地说 是伴随算子 adjoint)，我们需要先定义函数  $x(t)$  与  $y(t)$  的内积。

$$x^T y = (x, y) = \int_{-\infty}^{\infty} x(t)y(t) dt.$$

因此，内积从有限维的求和  $\sum x_k y_k$  变成了函数的积分。

—

矩阵的转置满足

$$(Ax, y) = (x, A^T y).$$

对于  $A = \frac{d}{dt}$ ，我们有

$$(Ax, y) = \int_{-\infty}^{\infty} \frac{dx}{dt} y(t) dt = \int_{-\infty}^{\infty} x(t) \left(-\frac{dy}{dt}\right) dt = (x, A^T y).$$

这里用了分部积分法 (integration by parts)。在这个过程中，导数从第一个函数  $x(t)$  转移到第二个函数  $y(t)$ ，并且会产生一个负号。这告诉我们：\*\*导数的转置是它的相反数\*\*。

—

因此，导数算子是反对称的 (antisymmetric)：

$$A = \frac{d}{dt}, \quad A^T = -\frac{d}{dt}.$$

对称矩阵满足  $S^T = S$ ，而反对称矩阵满足  $A^T = -A$ 。线性代数中也包含微分与积分 (见第 8 章)，因为它们都是线性算子。



套用分部积分公式：

$$\int u'(t) v(t) dt = \left[ u(t) v(t) \right]_{-\infty}^{\infty} - \int u(t) v'(t) dt.$$

这里令

$$u(t) = x(t), \quad u'(t) = \frac{dx}{dt}, \quad v(t) = y(t).$$

那么：

$$\int \frac{dx}{dt} y(t) dt = \left[ x(t) y(t) \right]_{-\infty}^{\infty} - \int x(t) \frac{dy}{dt} dt.$$

如果假设边界项  $\left[ x(t) y(t) \right]_{-\infty}^{\infty} = 0$ （比如  $x(t), y(t)$  在无穷远处衰减到 0，或者它们满足合适的边界条件），那么就得到：

$$\int_{-\infty}^{\infty} \frac{dx}{dt} y(t) dt = - \int_{-\infty}^{\infty} x(t) \frac{dy}{dt} dt.$$

这就是书里写的：

$$\int_{-\infty}^{\infty} \frac{dx}{dt} y(t) dt = \int_{-\infty}^{\infty} x(t) \left( -\frac{dy}{dt} \right) dt.$$

它说明了：导数算子的伴随（转置）就是带一个负号的导数算子。

### 命题 2.1

微分的反对称也可以应用到中心差分矩阵：

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad \text{转置成} \quad A^T = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = -A$$

前向差分矩阵的转置会变成反向差分矩阵，相当于乘  $-1$ 。在微分方程式中，第二导数（加速度）是对称的，第一导数（正比于速度的阻尼）是反对称的。

#### $A^T A$ 的含义 1. 代数性质

- 对称性： $(A^T A)^T = A^T A$ ;
- 半正定性：

$$x^T (A^T A) x = (Ax)^T (Ax) = \|Ax\|^2 \geq 0,$$

因此特征值均为非负。

2. 几何意义  $A: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，而  $A^T A: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 。它的作用可以理解为：先通过  $A$  把向量映射到输出空间，再通过  $A^T$  拉回输入空间。因此  $A^T A$  定义了一种新的“长度度量”：

$$\|x\|_A^2 := x^T (A^T A) x = \|Ax\|^2.$$

#### 3. 最小二乘中的作用正规方程

$$A^T A \hat{x} = A^T b$$

说明  $A^T A$  扮演了 Gram 矩阵的角色，它把“残差最小化”的几何条件转化为代数方程。

4. 数据分析中的意义  $A^T A$  又称 Gram 矩阵或协方差矩阵（标准化后）。它记录了列向量之间的内积关系：

$$(A^T A)_{ij} = a_i^T a_j,$$

并且是 PCA（主成分分析）等方法的核心。

**总结：**  $A^T A$  既是对称半正定的 Gram 矩阵，反映了列向量的相关性，又是最小二乘与 PCA 的核心工具。

**定理 2.3 (转置与内积的意义)**

$x^T y$  是一个数字,  $xy^T$  是一个矩阵.

在量子力学中常写作  $\langle x|y\rangle$  (内积) 与  $|x\rangle\langle y|$  (外积)。

宇宙可能是被线性代数管理的, 下面三个例子对于内积有不同的意义:

来自力学      功 (work) = (位移)(力)  $= x^T f$ ,

来自电路      热损失 = (压降)(电流)  $= e^T y$ ,

来自经济      收入 = (数量)(价格)  $= q^T p$ .

**定义 2.2 (列空间、行空间与零空间)**

还要再看设  $A \in \mathbb{R}^{m \times n}$ 。

- 列空间 (Column Space)

$$\mathcal{C}(A) = \{ Ax : x \in \mathbb{R}^n \} \subseteq \mathbb{R}^m.$$

它由  $A$  的列向量张成, 表示所有可能的输出  $Ax$ 。几何意义:  $A$  把  $\mathbb{R}^n$  中的向量映射到  $\mathbb{R}^m$ , 结果必定落在列空间中。

- 行空间 (Row Space)

$$\mathcal{C}(A^T) = \{ A^T y : y \in \mathbb{R}^m \} \subseteq \mathbb{R}^n.$$

它由  $A$  的行向量张成, 等价于  $A^T$  的列空间。几何意义: 行空间包含了  $A$  对输入向量  $x$  所进行的“线性约束”的所有可能组合。也就是  $Ax=b$  里面每一行  $x$  的系数限制  $x$  之间的关系。

- 零空间 (Null Space)

$$\mathcal{N}(A) = \{ x \in \mathbb{R}^n : Ax = 0 \}.$$

它是所有被  $A$  映射到零向量的输入向量的集合。几何意义: 零空间表示了“失效方向”——这些方向上的输入完全被  $A$  消去。

**命题 2.2 (三者对比)**

- 列空间  $\mathcal{C}(A) \subseteq \mathbb{R}^m$ , 行空间  $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$ , 零空间  $\mathcal{N}(A) \subseteq \mathbb{R}^n$ 。
- 维数关系:

$$\dim \mathcal{C}(A) = \dim \mathcal{C}(A^T) = \text{rank}(A).$$

设  $A \in \mathbb{R}^{m \times n}$ , 那么

$$\dim \mathcal{C}(A) + \dim \mathcal{N}(A) = n.$$

- 正交关系: 零空间  $\mathcal{N}(A)$  与行空间  $\mathcal{C}(A^T)$  正交; 零空间  $\mathcal{N}(A^T)$  与列空间  $\mathcal{C}(A)$  正交。



## 2.2 最小二乘法

$Ax = b$  经常是无解, 一般的理由是方程式太多。矩阵  $A$  的行比列多, 方程式的个数大于未知数的个数 ( $m > n$ )。这  $n$  个列只生成了  $m$  维空间的一小部分, 除非所有的量测值都很完美, 否则  $b$  会落在  $A$  的列空间之外。消元法得到不可能存在的方程式然后停止, 但我们不能停止, 因为量测值必然包含杂讯。

**重复:** 我们不可能一直把误差  $e = b - Ax$  降为零; 当  $e = 0$  时,  $x$  是  $Ax = b$  的确切解。当  $e$  的长度尽可能地小,  $\hat{x}$  就是“最小二乘解” (least squares solution)。本段的目标是计算并使用  $\hat{x}$ , 许多实际问题都需要它。

前一段强调投影  $p$ , 本段强调最小二乘解  $\hat{x}$ ; 二者通过  $p = A\hat{x}$  相关联。基本方程式仍然是正规方程:  $A^T Ax = A^T b$ 。下面给出一个非正式的得到“正规方程式”的办法:

 **笔记** 当  $Ax = b$  无解时, 左乘  $A^T$  然后求解

$$A^T Ax = A^T b.$$

我们希望用一条直线

$$y \approx ax + b$$

来拟合一组点。将参数写成向量

$$\theta = \begin{bmatrix} a \\ b \end{bmatrix},$$

并将数据点  $(x_i, y_i)$  写成矩阵形式:

$$A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

于是所有观测点可以统一写作

$$A\theta \approx y,$$

即

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax_1 + b \\ ax_2 + b \\ \vdots \\ ax_m + b \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

由于一般情况下方程组无解, 我们改为最小二乘问题:

$$\hat{\theta} = \arg \min_{\theta} \|A\theta - y\|^2,$$

其解为正规方程

$$\hat{\theta} = (A^T A)^{-1} A^T y.$$

### 东邪的 tip 2.3 (数值分析课上关于最小二乘法的原理)

这是经典的最小二乘误差函数:

$$E(a_0, a_1) = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2.$$

用导数方法求最小值 (原理关键!)

我们令误差函数对  $a_0, a_1$  的偏导数为 0, 找极小值点:

$$\frac{\partial E}{\partial a_0} = 0, \quad \frac{\partial E}{\partial a_1} = 0.$$

这是一个标准的“求极值”步骤。展开以后可以得到两个线性方程:

$$\begin{cases} na_0 + a_1 \sum x_i = \sum y_i, \\ a_0 \sum x_i + a_1 \sum x_i^2 = \sum x_i y_i. \end{cases}$$

直线拟合问题写成矩阵形式：

$$A\theta \approx y, \quad A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix}, \quad \theta = \begin{bmatrix} a \\ b \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

因为  $m > 2$  一般无解，改为最小二乘，得到正规方程

$$A^T A \theta = A^T y,$$

即

$$\begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & m \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix}.$$

于是得到两条方程：

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i, \\ a \sum x_i + b m = \sum y_i, \end{cases}$$

其解  $(a, b)$  正是使残差平方和最小的拟合直线参数。

### 命题 2.3 (从矩阵理解和从导数理解的关系)

我们要最小化的目标函数是

$$E(\theta) = \|A\theta - y\|^2 = (A\theta - y)^T (A\theta - y),$$

这是最小二乘的定义。

#### 2. 求偏导 = 0

对  $\theta$  求梯度：

$$\nabla_{\theta} E = 2A^T (A\theta - y).$$

设导数 = 0，则

$$A^T (A\theta - y) = 0,$$

这就得到正规方程

$$A^T A \theta = A^T y.$$

因此，“偏导数 = 0”推出的条件，和“左乘  $A^T$  保证残差正交”其实是同一个式子。

### 命题 2.4 (如何从投影理解优化目标)

矩阵  $A$  的所有列向量可以生成一个列空间，向量  $Ax$  一定在这个列空间内，但向量  $b$  可能不在列空间内。 $Ax$  与  $b$  的差就是一个新的向量  $r$ ，称为残差，可以用平行四边形法则理解。为了使  $r$  的长度最短，必须要求它垂直于由  $A$  的列向量生成的空间。如果满足这个条件，就有

$$A^T r = 0,$$

此时残差的长度也达到最小。几何上可以想象为：空间外一点到平面的最短距离就是垂直距离，而把  $b$  投影到由  $A$  的列向量张成的平面上后，就存在相应的  $x$ ，使得  $Ax$  恰好等于这个投影向量。

### 东邪的 tip 2.4

从几何，代数和微积分都能理解这个方程的目的。没多一种维度你对问题的理解就更加深刻。

**东邪的 tip 2.5**

矩阵乘法意义

一句话：谁在左边，谁就“主导”最后结果落在哪个空间；右边的对象只决定“怎么组合”，因此，当我们同时左乘  $A^T$  时，等式的两边都被映射到了  $A^T$  所作用的空间中。选择左乘  $A^T$  不是随意的，而是因为它唯一能把几何条件  $r \perp C(A)$ （残差垂直列空间）翻译成代数形式。

列空间与行空间

$$(Ax) \cdot y = x \cdot (A^T y)$$

$A$  生成的空间是“输出空间里的列空间”——列向量为基生成的空间， $A^T$  生成的空间是“输入空间里的行空间”——行向量生成空间。它们的维数相等，但位置不同，通过点积保持联系。所以，输入空间就是  $\mathbb{R}^n$ （因为你只能送  $n$  维向量进去）也就是乘的  $x$  必须是一个  $n$  维列向量，输出空间就是  $\mathbb{R}^m$ （因为出来的永远是  $m$  维）。

## 2.3 QR 分解操作

QR 分解的核心目的是将矩阵  $A$  分解为

$$A = QR,$$

其中  $Q$  是列向量正交归一的矩阵， $R$  是上三角矩阵。换句话说，QR 分解就是把原来可能不正交的基向量转化为一组正交单位向量，并将变化过程中的比例关系记录在  $R$  中。和前面的矩阵乘法概念类似，能清晰的看出  $R$  对原本的正交基有怎么样的拉伸和旋转。

**命题 2.5**

当  $Q$  是方阵且  $m = n$  时，子空间就是整个空间。此时有

$$Q^T = Q^{-1},$$

并且

$$\hat{x} = Q^T b \quad \text{与} \quad x = Q^{-1} b$$

完全相同，因此这个解是确切的。

此外，向量  $b$  在整个空间上的投影就是它自身：

$$p = b,$$

并且投影矩阵满足

$$P = Q^T Q = I.$$

这样我们从原来的两边同乘  $A^T$  变成两边同乘  $Q^T$  乘  $A^T A$  的方法会放大条件数（奇异值变成平方），误差更大。• 用  $Q^T$  投影不会改变长度或角度（正交变换保持数值稳定）。 $R$  作为上三角矩阵天然好解方程。

**命题 2.6 (正交矩阵与傅立叶变换)**

当  $p = b$ ，我们的公式把  $b$  从它的一维投影中聚集起来。若  $q_1, \dots, q_n$  是整个空间的一组正交单位基底，则  $Q$  是方形，每个  $b = QQ^T b$  是它沿着  $q$ 's 的分量的总和：

$$b = q_1(q_1^T b) + \dots + q_n(q_n^T b).$$

转换  $QQ^T = I$  是傅里叶级数的基础，也是所有应用数学中伟大“转换”的基础。他们把  $b$  或函数  $f(x)$  打碎成垂直的片段，借由上式中的片段，逆转换把  $b$  与  $f(x)$  再次一起恢复。

**东邪的 tip 2.6 (施密特正交化)**

看见转置要想到点积，看见点积要想到投影。如何把三个向量  $a, b, c$  正交化呢？方法就是：把每一个向量在其他向量方向上的分量减掉。

格莱姆-施密特的第一步：

$$B = b - \frac{A^T b}{A^T A} A, \quad (7)$$

其中投影系数为

$$\frac{\langle b, A \rangle}{\langle A, A \rangle} = \frac{A^T b}{A^T A}.$$

两边左乘  $A$  的转置即可验证  $A$  与  $B$  的正交。 $c$  同理去掉  $a$  和  $B$  的分量产生  $C$ 。最后  $a, B, C$  再除以自己的模长即可。一句话总结

Gram-Schmidt 正交化 = 把  $A$  的列向量逐个「扣掉已有正交方向的分量」。这个扣除的系数恰好填入  $R$  的上三角位置，剩下的单位化结果填入  $Q$  的列。 $A = QR$  不是额外的结论，而是 Gram-Schmidt 过程的直接重写。

**命题 2.7 (格莱姆-施密特的关键步骤)**

第一步是

$$q_1 = \frac{a}{\|a\|},$$

(其他向量不参与)。第二步是方程式 (7)，其中  $b$  是  $A$  与  $B$  的组合，在这一步骤  $c$  与  $q_3$  没有参与。下一个向量不参与正是格莱姆-施密特的关键点：这就是为什么下三角可以直接写 0。

- 向量  $a$  与  $A$  与  $q_1$  沿着同一条直线；
- 向量  $a, b$  与  $A, B$  与  $q_1, q_2$  在同一个平面；
- 向量  $a, b, c$  与  $A, B, C$  与  $q_1, q_2, q_3$  在同一个子空间（维度 3）。

在每一步， $a_1, \dots, a_k$  是  $q_1, \dots, q_k$  的组合，后面的  $q$  不参与。关联矩阵  $R$  是三角形，我们有

$$A = QR.$$

$$[a \ b \ c] = [q_1 \ q_2 \ q_3] \begin{bmatrix} q_1^T a & q_1^T b & q_1^T c \\ 0 & q_2^T b & q_2^T c \\ 0 & 0 & q_3^T c \end{bmatrix}, \quad \text{或简记为 } A = QR. \quad (9)$$

 **笔记** 设  $A$  的列向量是  $a_1, a_2, a_3$ ，做 Gram-Schmidt 正交化：

$$q_1 = \frac{a_1}{\|a_1\|}, \quad q_2 = \frac{a_2 - (a_2 \cdot q_1)q_1}{\|\dots\|}, \quad q_3 = \frac{a_3 - (a_3 \cdot q_1)q_1 - (a_3 \cdot q_2)q_2}{\|\dots\|}$$

反过来用  $q_i$  表示原列向量：

$$a_1 = \|a_1\|q_1, \quad a_2 = (a_2 \cdot q_1)q_1 + \|\dots\|q_2, \quad a_3 = (a_3 \cdot q_1)q_1 + (a_3 \cdot q_2)q_2 + \|\dots\|q_3$$

写成矩阵形式即  $A = QR$ ：

$$\begin{bmatrix} | & | & | \\ a_1 & a_2 & a_3 \\ | & | & | \end{bmatrix} = \begin{bmatrix} | & | & | \\ q_1 & q_2 & q_3 \\ | & | & | \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

$R$  为何上三角？因为  $a_k$  正交化时只涉及前  $k$  个基向量，第  $k$  列仅  $r_{1k}, \dots, r_{kk}$  非零，天然产生上三角结构。

**定理 2.4 (格莱姆-施密特正交化)**

从无关向量  $a_1, \dots, a_n$  出发, 格莱姆-施密特方法能够构造出正交单位向量  $q_1, \dots, q_n$ 。这些列向量构成的矩阵  $Q$  满足

$$A = QR,$$

其中

$$R = Q^T A$$

是一个上三角矩阵, 因为后面的  $q$  与较早的  $a$  的内积为零 (正交性)。



$$\text{最小二乘: } R^T R \hat{x} = R^T Q^T b \quad \text{或} \quad R \hat{x} = Q^T b \quad \text{或} \quad \hat{x} = R^{-1} Q^T b \quad (10)$$

## 2.4 QR 分解迭代法-数值分析

基本的步骤是分解  $A$  成为  $QR$ , 其中  $A$  的特征值是我们想要的。回忆格莱姆-施密特 (段落 4.4),  $Q$  有正交单位列与  $R$  是三角形。特征值的关键概念是: 反转  $Q$  与  $R$ 。新的矩阵 (相同  $\lambda$ 's) 是  $A_1 = RQ$ , 因为  $A = QR$  与  $A_1 = Q^{-1} A Q$  相似, 所以  $RQ$  的特征值没有改变:

$$A_1 = RQ \text{ 有相同的特征值 } \lambda, \quad QRx = \lambda x \Rightarrow RQ(Q^{-1}x) = \lambda(Q^{-1}x). \quad (12)$$

继续这个程序。把新矩阵  $A_1$  分解成  $Q_1 R_1$ , 然后反转因子变成  $R_1 Q_1$ , 这是相似矩阵  $A_2$  而且再次没有改变特征值。惊人的是这些特征值开始出现在对角线, 很快的  $A_4$  的最后单元得到一个精确的特征值。这个情况下, 我们移除最后一行与最后一列得到一个较小的矩阵, 然后继续寻找下一个特征值。

这个现象的原因是

$$A_k = (Q_1 Q_2 \cdots Q_k)^T A_0 (Q_1 Q_2 \cdots Q_k).$$

记累计的正交变换

$$Q^{(k)} := Q_1 Q_2 \cdots Q_{k-1} \quad (\text{正交}),$$

于是

$$A_k = (Q^{(k)})^T A_0 Q^{(k)}.$$

这说明: 幂迭代是找的一个特征向量, QR 迭代是找一组特征向量做成的正交基

**命题 2.8**

两个额外的概念使得这个方法成功, 一个是在分解成  $QR$  之前, 把矩阵平移  $I$  的倍数, 然后  $RQ$  再平移回去得到  $A_{k+1}$ :

$$A_k - c_k I = Q_k R_k, \quad A_{k+1} = R_k Q_k + c_k I.$$

于是  $A_{k+1}$  与  $A_k$  有相同的特征值, 而且与原始的  $A_0 = A$  有相同的特征值。一个优质的平移是选择  $c$  靠近一个 (未知的) 特征值。这个更精确的特征值出现在  $A_{k+1}$  的对角线上——这告诉我们下一个到  $A_{k+2}$  的步骤应该选择的较好  $c$ 。

—

第二个概念是在 QR 方法开始之前先获得非对角线的零, 一个消元步骤  $E$  会做这样的事情, 或者是一个吉文斯旋转 (Givens rotation)。但是不要忘了  $E^{-1}$  (否则  $\lambda$  会改变):

$$EAE^{-1} = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 1 & 4 \\ -1 & 1 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 7 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 5 & 3 \\ 1 & 9 & 5 \\ 0 & 4 & 2 \end{bmatrix} \quad (\text{相同的 } \lambda\text{'s}).$$

我们必须在次对角线留下非零数 1 与 4，更多的  $E$  可以移除它们，但是  $E^{-1}$  会再次填满。这是一个海森堡矩阵（一个非零的次对角线）。

在 QR 方法过程中，左下角的零仍然维持是零。因此，每个 QR 分解的运算次数可以从  $O(n^3)$  降到  $O(n^2)$ 。

### 命题 2.9 (幂迭代与逆迭代)

最大特征值对应的向量会占据主导地位因为同时提出  $\lambda_1^k$ ，除了最大的一项，其他的  $\lambda_n/\lambda_1$  比值都小于 1， $k$  趋紧无穷的时候他们都趋紧于 0。

从任意向量  $u_0$  开始。用  $A$  乘它得到  $u_1$ 。再用  $A$  乘，得到  $u_2$ 。如果  $u_0$  是特征向量的线性组合，那么  $A$  乘以每个特征向量  $x_i$  时会放大  $\lambda_i$  倍。经过  $k$  步之后，我们得到  $(\lambda_i)^k$ ：

$$u_k = A^k u_0 = c_1(\lambda_1)^k x_1 + \cdots + c_n(\lambda_n)^k x_n. \quad (10)$$

随着幂迭代的进行，最大的特征值会逐渐占据主导地位。向量  $u_k$  会逐渐指向那个主特征向量  $x_1$ 。我们在马尔可夫矩阵中看到过这一点：

$$A = \begin{bmatrix} 0.9 & 0.3 \\ 0.1 & 0.7 \end{bmatrix} \quad \text{其最大特征值 } \lambda_{\max} = 1, \text{ 对应的特征向量为 } \begin{bmatrix} 0.75 \\ 0.25 \end{bmatrix}.$$

从  $u_0$  开始，并在每一步都用  $A$  相乘：

$$u_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad u_1 = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} 0.84 \\ 0.16 \end{bmatrix},$$

逐渐趋向于

$$u_{\infty} = \begin{bmatrix} 0.75 \\ 0.25 \end{bmatrix}.$$

收敛速度取决于第二大特征值  $\lambda_2$  与最大特征值  $\lambda_1$  的比值。我们并不希望  $\lambda_1$  太小，而是希望  $\lambda_2/\lambda_1$  较小。在这个例子中， $\lambda_2 = 0.6$ ， $\lambda_1 = 1$ ，因此收敛速度很好。

对于大矩阵，往往会发生  $|\lambda_2/\lambda_1|$  很接近 1 的情况，这时幂迭代就太慢了。

有没有办法找到最小的特征值（这往往在应用中最重要）呢？答案是肯定的：用反幂迭代（inverse power method）：用  $A^{-1}$  而不是  $A$  去乘  $u_0$ 。由于我们从不真的去计算  $A^{-1}$ ，实际上我们解的是  $Au_1 = u_0$ 。通过保存  $LU$  分解，下一步  $Au_2 = u_1$  也可以快速完成。在第  $k$  步时，有：

$$Au_k = u_{k-1}. \\ u_k = A^{-k} u_0 = \frac{c_1 x_1}{(\lambda_1)^k} + \cdots + \frac{c_n x_n}{(\lambda_n)^k}. \quad (11)$$

现在，最小的特征值  $\lambda_{\min}$  起主导作用。当它非常小时，因子  $1/\lambda_{\min}^k$  就会很大。为了加快速度，我们通过把矩阵平移为  $A - \lambda^* I$  来使  $\lambda_{\min}$  变得更小。

这种平移不会改变特征向量。 $(\lambda^*$  可以取自  $A$  的对角元, 更好的选择是 Rayleigh 商

$$\frac{x^T Ax}{x^T x}.$$

)

如果  $\lambda^*$  接近  $\lambda_{\min}$ , 那么  $(A - \lambda^* I)^{-1}$  就有一个非常大的特征值  $(\lambda_{\min} - \lambda^*)^{-1}$ 。

每一步 平移反幂迭代都会把该特征向量乘以这个大数, 从而使该特征向量迅速占据主导地位。

## 第3章 LLL 格基

### 定义 3.1 (格与格基)

设  $b_1, b_2, \dots, b_n \in \mathbb{R}^m$  是  $n$  个线性无关向量。由它们生成的格定义为

$$L(b_1, b_2, \dots, b_n) = \left\{ z_1 b_1 + z_2 b_2 + \dots + z_n b_n \mid z_i \in \mathbb{Z} \right\}.$$

称  $B = (b_1, b_2, \dots, b_n)$  为格  $L(B)$  的格基。注意，一个格可以有多种不同的基。



**注** 与普通的向量空间基不同，格基规定系数必须是整数：

$$L(B) = \{ z_1 b_1 + \dots + z_n b_n \mid z_i \in \mathbb{Z} \}.$$

因此，格是离散的点集，而不是连续的空间。

### 命题 3.1 (LLL 算法的核心思想)

LLL 算法不能追求完全正交（那样就不再是格基），只能在“整数保持”和“尽量正交”之间折中：

- 保证：所有变换都是整系数可逆矩阵（unimodular），因此格  $\mathcal{L}(B)$  保持不变；
- 目标：基向量不要太“偏斜”，做到“近似正交”，同时“尽量短”。



## 输入与目标

设格基为

$$B = (b_1, b_2, \dots, b_n) \in \mathbb{Z}^{m \times n}.$$

目标：通过基变换得到“更短、更接近正交”的基。其中每一个向量里面全是整数，为最后的势函数  $(B)$  提供约束。

$$B' = (b'_1, b'_2, \dots, b'_n),$$

满足一定的正规化条件（LLL-reduced）。

## Gram-Schmidt 正交化

记 Gram-Schmidt 结果：

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{j,i} b_j^*, \quad \mu_{j,i} = \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}.$$

于是  $b_i^*$  是  $b_i$  在  $\text{span}\{b_1, \dots, b_{i-1}\}$  的正交补上的投影。

### 东邪的 tip 3.1

$\mu_{j,i}$  是  $b_i$  中含有  $\mu_{j,i}$  倍的  $b_j$ ，如果这个系数很大，说明  $b_i$  在  $b_j^*$  方向上“粘连”得厉害  $\rightarrow$  基向量会偏斜。

## 步骤一：规模约化 (Size Reduction)

$$b_i \leftarrow b_i - \lfloor \mu_{j,i} \rfloor b_j, \quad \text{for } j < i,$$

其中  $\lfloor \cdot \rfloor$  表示四舍五入到最近的整数。不断减去整数倍的前面向量，避免“过度粘连”，并且保证点仍在格内。让系数  $\mu_{j,i}$  控制在

$$|\mu_{j,i}| \leq \frac{1}{2}.$$

这保证了基向量不会“偏斜”，得到更短正交。

### 定义 3.2 (Unimodular 矩阵)

一个整数矩阵  $U \in \mathbb{Z}^{n \times n}$  若满足

$$\det(U) = \pm 1,$$

则称为 *unimodular* 矩阵。



### 命题 3.2 (Unimodular 矩阵及其作用)

若  $U \in \mathbb{Z}^{n \times n}$  且  $\det(U) = \pm 1$ ，则称  $U$  为 *unimodular* 矩阵。它有以下性质：

1. 保持格不变. 若  $B$  是格基，则

$$\mathcal{L}(B) = \{Bz : z \in \mathbb{Z}^n\}, \quad B' = BU,$$

则有

$$\mathcal{L}(B') = \mathcal{L}(B).$$

因为  $U$  可逆且整系数，保证  $\mathbb{Z}^n$  在变换下仍映射回  $\mathbb{Z}^n$ 。

2. 保持体积. 平行多面体体积满足

$$\det(BU) = \det(B) \det(U) = \pm \det(B),$$

因此  $|\det(B)|$  保持不变。

3. LLL 算法中的意义. 在 LLL 算法中，所有基向量的调整都可表示为右乘 unimodular 矩阵：

- 规模约化 (Size Reduction)：

$$b_i \leftarrow b_i - \lfloor \mu_{i,j} \rfloor b_j, \quad j < i,$$

对应于右乘一个整系数初等矩阵，保证  $\mu_{i,j}$  被控制在  $[-\frac{1}{2}, \frac{1}{2}]$ 。

- 交换步骤 (Swap)：

$$b_i \leftrightarrow b_{i-1},$$

对应于交换两列，也就是 unimodular 矩阵。

这些操作保证始终在同一个格中进行换基，而不会改变格的本质。

因此，unimodular 矩阵就是“合法的换基操作”：它能改变基向量的形状，但不会改变格本身，也不会改变体积。



## 步骤二：Lovász 条件与交换

在规模约化后，检查相邻两列是否满足 Lovász 条件：

$$\|b_i^*\|^2 \geq (\delta - \mu_{i-1,i}^2) \|b_{i-1}^*\|^2, \quad \delta \in \left(\frac{1}{4}, 1\right),$$

通常取  $\delta = \frac{3}{4}$ 。

- Lovasz 条件大致保证当前工作的向量足够大成为我们的临时基向量  
在 Gram-Schmidt 正交化中，一个基向量  $b_i$  会被投影到前面的正交补  $b_i^*$  上。
    - 如果  $b_i^*$  太短，说明  $b_i$  在方向上几乎被前面基向量“解释掉了”；
    - 那么整个基就很“塌”，缺乏独立性，导致表示格点的效率很差。
- 于是 LLL 算法要求：

$\|b_i^*\|^2$  不能太小，相对前一个正交向量  $b_{i-1}^*$  而言要有下界。

- 若条件成立：继续处理  $i+1$  列；
- 若条件不成立：交换  $b_{i-1}$  与  $b_i$ ，并令  $i \leftarrow \max(i-1, 2)$ ，然后重新规模约化。

### 命题 3.3 (LLL 算法的交换思想)

在施密特正交化中，按顺序处理基向量：前面的基先正交化，后面的基不断减去前面的投影。LLL 的交换条件 (Lovász 条件) 就是在检查：

- 前面的基向量是否“太差劲” (太长或太偏斜)，
- 如果条件不满足，就交换  $b_k$  和  $b_{k+1}$ 。

这样做的结果是：

- 较短、方向更合适的向量被提前，作为参照基；
- 较差的向量被放到后面，被迫减去“好的”向量的分量。

**Lovász 条件：**

$$\delta \|b_k^*\|^2 \leq \|b_{k+1}^* + \mu_{k+1,k} b_k^*\|^2,$$

其中  $b_i^*$  是施密特正交向量， $\delta \in (1/4, 1)$  (通常取  $\delta = 3/4$ )。

- 若条件成立，说明  $b_k$  “够好”，无需交换；
- 若条件不成立，说明  $b_k$  太差，就与  $b_{k+1}$  交换。

类比：可以把这一过程看作“选队长”：

- 好的队员 (短而正交性好) 先当队长；
- 差的队员只能排在后面，被迫去配合前面更强的队员。

这样，整个队伍 (格基) 就会更整齐、更短、更接近正交。

### 命题 3.4 (LLL 与冒泡排序的类比)

在冒泡排序中，较大的元素 (若从大到小排序) 会一步步“冒”到前面：

- 每次比较相邻两个，如果顺序错误，就交换。
- 经过多轮，最终最“大”的元素会被送到最前面。

在 LLL 算法中，同样存在类似的“冒泡”现象：

- LLL 希望“更短、更正交”的向量排在前面。
- 每次检查 Lovász 条件

$$\delta \|b_k^*\|^2 \leq \|b_{k+1}^* + \mu_{k+1,k} b_k^*\|^2,$$

若不满足，就交换  $b_k \leftrightarrow b_{k+1}$ 。

- 交换后，那个“更好”的向量 (更短、更正交的) 会向前移动一步，就像冒泡排序里较大的元素逐步向前冒。
- 因此，经过不断交换，“最好的基向量”会逐渐被推到前面。

### 东邪的 tip 3.2 (类比：LLL 的 Lovász 条件 vs ODE 的 Lipschitz 条件)

| LLL 格基约化                                                                                 | 常微分方程 (ODE)                                 |
|------------------------------------------------------------------------------------------|---------------------------------------------|
| Lovász 条件：保证相邻正交向量不能“太塌”                                                                 | Lipschitz 条件：保证函数变化不能“太陡”                   |
| $\delta \ b_k^*\ ^2 \leq \ b_{k+1}^* + \mu_{k+1,k} b_k^*\ ^2, \quad \delta \in (1/4, 1)$ | $ f(x, y_1) - f(x, y_2)  \leq L y_1 - y_2 $ |
| 避免基向量几乎线性相关，保证格的稳定性                                                                      | 避免解不唯一，保证 ODE 解的稳定性                         |

## 步骤三：终止性与复杂度

定义一个势函数

$$\Phi(B) = \prod_{i=1}^n \|b_i^*\|^2,$$

这个势函数就所有  $b_i$  模长的平方的乘积。它在每次交换后都会严格减小一个有界的整数量，因此算法在有限步后必然终止。

复杂度结果：LLL 在输入长度的多项式时间内完成（这是它在密码学应用中的关键优势）。

### 命题 3.5 (LLL 势函数与格体积)

设  $B = [b_1, \dots, b_n] \in \mathbb{R}^{m \times n}$ ，Gram-Schmidt 正交化得  $b_1^*, \dots, b_n^*$ 。则

$$\det(B^T B) = \prod_{i=1}^n \|b_i^*\|^2.$$

因此定义的势函数

$$\Phi(B) := \prod_{i=1}^n \|b_i^*\|^2$$

正是格体积的平方。

由于 LLL 算法中的变换均为 unimodular 矩阵作用（保持体积不变）， $\Phi(B)$  始终与格的几何结构对应，并在交换或约化时严格下降，保证算法收敛。



### 命题 3.6 (有限步终止性的直观理解.)

在整格  $\mathcal{L}(B) \subseteq \mathbb{Z}^m$  中，任意一组基  $B$  生成的平行多面体

$$P(B) = \left\{ \sum_{i=1}^n \alpha_i b_i : 0 \leq \alpha_i < 1 \right\}$$

的体积必为整数，因为它正好对应  $\mathbb{R}^m$  中由单位立方格子堆叠出的“积木”个数。因此

$$\det(\mathcal{L}(B)) = |\det(B)| \in \mathbb{Z}_{>0}.$$

Gram-Schmidt 分解给出

$$\prod_{i=1}^n \|b_i^*\| = |\det(B)|,$$

于是势函数

$$\Phi(B) = \prod_{i=1}^n \|b_i^*\|^2 \in \mathbb{Z}_{>0}.$$

换句话说，每一个格基形成的平行多面体体积天然为整数，势函数就是它的“平方版”。每次交换都会严格减小  $\Phi(B)$ ，而  $\Phi(B)$  只有有限多个整数值，因此 LLL 算法必在有限步后终止。



### 东邪的 tip 3.3

离散的好处就是有限，至少相对连续来说好很多

## 结果性质

输出基  $(b'_1, \dots, b'_n)$  满足：

- 规模约化：  $|\mu_{j,i}| \leq \frac{1}{2}, \forall j < i$ ;
- Lovász 条件：  $\|b_i^*\|^2 \geq (\delta - \mu_{i-1,i}^2) \|b_{i-1}^*\|^2$ ;
- 基向量较短且接近正交。

## 3.1 LLL 最短向量的意义

用 LLL 从数值  $\alpha$  识别整系数多项式：以  $\alpha \approx \sqrt{2}$  为例

目标 已知一个实数的近似值  $\alpha$  (例如  $\alpha = 1.414213$ ), 希望找到一个整系数多项式

$$f(x) = c_0 + c_1x + \cdots + c_dx^d \in \mathbb{Z}[x]$$

使得  $|f(\alpha)|$  很小 (理想情况是  $f(\alpha) = 0$ , 即找出  $\alpha$  的最小多项式)。

格的构造 (How to build a lattice) 取一组尺度  $k \in \mathbb{N}$  (比如  $k = 6$ ), 并选择多项式次数上界  $d$ 。构造如下  $(d+1) \times (d+1)$  矩阵  $B$  的列向量作为格基:

$$B = \begin{bmatrix} 1 & & & 0 \\ & 1 & & 0 \\ & & \ddots & 0 \\ 0 & 0 & \cdots & 1 \\ 10^k & [10^k\alpha] & [10^k\alpha^2] & \cdots \end{bmatrix} \quad (\text{上方是 } I_{d+1}, \text{ 最后一行放 } 10^k\alpha^i \text{ 的取整——为了保证是整数 } k \text{ 的大小决定了精度}).$$

### 东邪的 tip 3.4

其中, 对角线上的元素依次代表  $x^0$  到  $x^{d-1}$ 。它们的线性组合所形成的每一列, 都对应于一组  $(1, x, x^2, \dots, x^{d-1})$  的系数。最后一行则是为了将这组线性组合的  $x$  直接代入  $\alpha$ , 使结果能够清晰直观地显示出来。我们的目的是让这组线性组合经可能的小最好是 0。这里还有一个放大的好处由于最后一行远大于前面的行所以最后一行对向量的大小起决定性作用, 最短的向量的最后一行必然是所有最后一行元素最小的。

\* 用 LLL 找“短向量”  $\Rightarrow$  找多项式系数 对基  $B$  运行 LLL 算法, 得到一个“较短且近似正交”的等价基。LLL 输出基中的最短向量或若干很短的向量, 其整数系数  $z = (c_0, \dots, c_d)$  往往满足  $|f(\alpha)| = |\sum c_i\alpha^i|$  很小。于是我们就得到了一个候选多项式

$$f(x) = c_0 + c_1x + \cdots + c_dx^d \in \mathbb{Z}[x], \quad \text{且 } |f(\alpha)| \text{ 很小.}$$

若  $f(\alpha) \approx 0$ , 可尝试化简 (去公因数) 并检验是否确为最小多项式。

具体例子:  $\alpha \approx \sqrt{2}, d = 2, k = 6$  取

$$\alpha = 1.414213, \quad d = 2, \quad k = 6.$$

构造

$$B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 10^6 & [10^6\alpha] & [10^6\alpha^2] \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1000000 & 1414213 & 2000000 \end{bmatrix} \quad (\text{因为 } \alpha^2 \approx 2, [10^6\alpha^2] \approx 2000000).$$

对该  $B$  做 LLL 约化, 得到的一个最短向量 (或很短向量) 可能是

$$(-2, 0, 1)^\top.$$

它对应的多项式为

$$f(x) = (-2) + 0 \cdot x + 1 \cdot x^2 = x^2 - 2,$$

并且

$$f(\alpha) = \alpha^2 - 2 \approx 0.$$

这正是  $\sqrt{2}$  的最小多项式。

总结

- 将“寻找整系数多项式  $f$  使  $f(\alpha)$  小”转化为“寻找格中的短向量”;
- 通过构造含  $I_{d+1}$  与  $[10^k\alpha^i]$  的基矩阵  $B$ , 使每个整数系数向量  $z$  对应一个多项式;
- 用 LLL 在多项式时间内找到短向量, 其系数即为候选多项式;

- 在  $\alpha \approx \sqrt{2}$  的例子中, LLL 返回  $(-2, 0, 1)$ , 对应  $x^2 - 2$ 。

如果生成的不是最短多项式就是小数点后面精度不够。最短多项式就是, 项数最少的多项式, LLL 查找不同数字间的整数关系。整数本身就是一种简化。

### 例题 3.1 Merkle-Hellman 背包加密示例

#### 1. 密钥生成

选择一个超递增序列:

$$w = (2, 3, 7, 14, 30).$$

取模数  $q = 61$  (大于  $\sum w_i = 56$ ), 并选取  $r = 17$  与  $q$  互素。

计算公钥:

$$b_i \equiv r \cdot w_i \pmod{q},$$

得到

$$b = (34, 51, 58, 54, 22).$$

因此: - 私钥为  $(w, q, r)$ ; - 公钥为  $b = (34, 51, 58, 54, 22)$ 。

明文消息为比特串  $(1, 0, 1, 0, 1)$ 。

计算密文:

$$c = \sum_{i=1}^5 m_i b_i = 1 \cdot 34 + 0 \cdot 51 + 1 \cdot 58 + 0 \cdot 54 + 1 \cdot 22 = 114.$$

因此密文为:

$$c = 114.$$

#### 3. 解密

首先计算  $r$  在模  $q$  下的逆元:

$$r^{-1} \equiv 18 \pmod{61}, \quad \text{因为 } 17 \cdot 18 \equiv 1 \pmod{61}.$$

然后:

$$c' \equiv c \cdot r^{-1} \pmod{q} = 114 \cdot 18 \equiv 37 \pmod{61}.$$

在超递增序列  $w = (2, 3, 7, 14, 30)$  中解方程

$$\sum_{i=1}^5 m_i w_i = 37.$$

使用贪心法:  $-30 \leq 37 \implies m_5 = 1$ , 余数 7;  $-14 > 7 \implies m_4 = 0$ ;  $-7 = 7 \implies m_3 = 1$ , 余数 0; - 余下均为 0。

因此恢复的明文为:

$$(m_1, m_2, m_3, m_4, m_5) = (1, 0, 1, 0, 1).$$

**注** Merkle-Hellman 背包加密体系最初被认为是安全的, 因为从公钥  $b = (b_1, \dots, b_n)$  和密文  $c = \sum m_i b_i$  恢复比特向量  $(m_1, \dots, m_n)$  等价于求解

$$\sum_{i=1}^n m_i b_i = c, \quad m_i \in \{0, 1\},$$

这正是著名的 子集和问题 (subset sum problem), 在一般情况下是 NP-困难的。

然而, LLL 格基约化算法提供了一种有效的攻击途径: 通过构造一个  $(n+1)$  维格, 其中包含向量

$$(a_1, \dots, a_n, c),$$

LLL 可以在多项式时间内找到短向量。这些短向量往往正好对应于满足上述子集和关系的比特向量  $(m_1, \dots, m_n)$ 。

因此，原本依赖于子集和难解性的 Merkle–Hellman 背包体系，在 LLL 算法出现之后不再安全，很快被实际攻破。这也是格密码学对现代密码体系影响的重要历史事件之一。

### 定义 3.3 (时间复杂度)

在计算机科学中，算法的时间复杂度是指：算法在最坏情况或平均情况下所需的基本运算次数，作为输入规模  $n$  的函数。换句话说，时间复杂度研究的是当  $n$  越来越大时，算法运行时间如何随  $n$  增长。

设算法  $A$  在输入规模为  $n$  时，所需的基本操作次数为  $T(n)$ 。如果存在常数  $c > 0$  和  $n_0$ ，使得对所有  $n \geq n_0$  都有

$$T(n) \leq c \cdot f(n),$$

则称该算法的时间复杂度是  $O(f(n))$ 。其中  $f(n)$  是一个常见的函数，例如  $n, n^2, n \log n$  等。



## 3.2 LLL 的时间复杂度

### 定理 3.1 (LLL 算法的时间复杂度 (经典结论))

设输入是  $n$  维整数格的基  $B \in \mathbb{Z}^{n \times n}$ ，其元素绝对值不超过  $B$ 。对固定的  $\delta \in (1/4, 1)$ ，标准 LLL 算法在位操作意义下的时间复杂度满足

$$T(n, B, \delta) = \tilde{O}(n^4 \cdot \log B),$$

更精细的经典上界常写作

$$O(n^4 \cdot \log^3 B),$$

其中  $\tilde{O}$  省略了多项式对数因子。实际实现常优于理论上界。



GSO 就是施密特正交化

| 步骤   | 动作                                           | 单次代价     | 备注                                         |
|------|----------------------------------------------|----------|--------------------------------------------|
| 1    | 初始化 GSO (计算 $\mu_{i,j}, \ b_j^*\ ^2$ )       | $O(n^3)$ | 一次性操作                                      |
| 2    | 尺寸约化 (对 $j < i$ : 算 $\mu_{i,j}$ , 更新 $b_i$ ) | $O(n^2)$ | 内积 $O(n)$ + 向量更新 $O(n)$ , 共 $i-1 \leq n$ 次 |
| 3    | Lovász 条件检查                                  | $O(1)$   | 只用已存的 $\mu, \ b^*\ ^2$                     |
| 4    | 交换并更新 GSO (若条件不满足)                           | $O(n^2)$ | 增量更新避免 $O(n^3)$                            |
| 循环次数 | 交换步数 $\leq O(n^2 \log B)$                    | ——       | 势函数下降法保证                                   |
| 总复杂度 | $O(n^4 \log^3 B)$ (其中 $\log^3 B$ 来自大整数算术位长)  |          |                                            |

表 3.1: LLL 算法每步操作及时间复杂度

### 定理 3.2 (LLL 算法的复杂度上界)

设输入基为  $B \in \mathbb{Z}^{n \times n}$ ，其元素的绝对值不超过  $B$ 。则 Lenstra–Lenstra–Lovász (1982) 的分析表明：

- 所有中间数的位长增长是多项式有界，不会发生指数爆炸；
- 在保守估计下，每次算术操作的位复杂度需要额外考虑  $(\log B)^2$  或  $(\log B)^3$ 。

因此，LLL 算法的总时间复杂度满足

$$T(n, B) = O(n^4 \cdot \log^3 B).$$



### 命题 3.7 (对 $\log^3 B$ 的拆解理解)

复杂度中的  $\log^3 B$  可以解释为以下三个因素的叠加：

1. 第一个  $\log B$ ：来自输入规模，即最大数  $B$  的比特长度；
2. 第二个  $\log B$ ：来自大整数算术运算（乘法、除法）的代价；

3. 第三个  $\log B$ : 来自迭代过程中中间数的膨胀, 虽数值变大但仍在多项式范围内。  
因此最终得到  $\log^3 B$ 。



## 第4章 基于树的方法

### 4.1 回归树

#### 定义 4.1 (回归树)

回归树是一种非参数化的监督学习方法，用于预测连续型输出变量。它通过递归地划分输入特征空间，并在每个划分区域内用常数值（通常是该区域训练样本的均值）作为预测，从而构建一个分段常数函数近似目标函数。

建立回归树的过程大致可以分为两步：

1. 将预测变量空间（即  $X_1, X_2, \dots, X_p$  的可能取值构成的集合）分割成  $J$  个互不重叠的区域  $R_1, R_2, \dots, R_J$ 。
2. 对落入区域  $R_j$  的每个观测值做同样的预测，预测值等于  $R_j$  上训练集的响应值的简单算术平均。

举个例子，若在第一步中得到两个区域  $R_1$  和  $R_2$ ， $R_1$  上训练集的平均响应值为 10， $R_2$  上训练集的平均响应值为 20。那么，对于给定的观测值  $X = x$ ，若  $x \in R_1$ ，则预测值为 10；若  $x \in R_2$ ，则预测值为 20。

现在详细说明上面的第一步。如何构建区域  $R_1, R_2, \dots, R_J$ ？理论上，区域的形状是任意的，但出于模型简化和增强可解释性的考虑，这里将预测变量空间划分成高维矩形，或称盒子（box）。划分区域的目标是找到使模型的残差平方和（RSS）最小的矩形区域  $R_1, R_2, \dots, R_J$ 。

RSS 的定义为

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2.$$

上式中的  $\hat{y}_{R_j}$  是第  $j$  个矩形区域中训练集的平均响应值。

#### 东邪的 tip 4.1 (回归树的原理)

回归树相当于一个预处理，先把具有相同规律的点的区域找出来，然后与其他区域区别开。当我们要预测的时候只需要判断点落在那个区域然后预测他为该区域平均数即可。RSS 的大小就是显示我们的刀法是不是精准，就是所有区域的误差加起来。用每一个区域的点减去该区域的平均值的平方和是为了放大误差。回归树的本质可以概括为：

- 回归树 = 把点分区块，每块用一个小模型（常数函数）解释数据；
- 它是一个分段常数函数的几何直观。

回归树的直观理解

1. **数据点视角** 假设你有一堆数据点  $(x_i, y_i)$ ，比如  $x$  是房子的面积、房间数， $y$  是房价。把这些点放到坐标系里，就是数据分布的样子。
  2. **划分区域** 回归树做的事就是：不断选择一个特征（例如“面积”），找一个切分点（例如 120 平方米），把数据分成两个区域。然后在每个区域里，再继续切（比如在“房间数”上切一刀），直到区域足够“纯净”。最终数据空间被切成一个个矩形/盒子（box）。
  3. **区域内的模型** 在每个区域  $R_j$  内，用一个常数模型来解释数据：就是该区域里所有  $y$  的平均值。所以区域越“纯”，这个平均值越能代表里面的点。预测时：一个新点  $x$  落在哪个区域，就直接输出该区域的平均  $y$ 。
- 举个例子

假如我们用回归树预测房价：

- 切分 1：面积  $\leq 120$  平  $\Rightarrow$  走左边区域，否则走右边。
- 切分 2（左区域里）：房间数  $\leq 2 \Rightarrow$  预测 100 万，否则预测 150 万。
- 切分 3（右区域里）：面积  $> 200 \Rightarrow$  预测 300 万，否则预测 220 万。

结果就是把整个房子数据分成几块，每块用一个平均值解释。

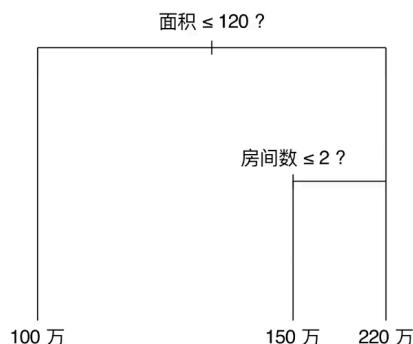


图 4.1: 回归树的结构示意图：每个切分对应一个特征阈值，叶子节点给出预测值。

这也就是为什么叫树。

### 4.1.1 回归树的具体计算

遗憾的是，要想考虑将特征空间划分为  $J$  个矩形区域的所有可能性，在计算上是不可行的。因此一般采用一种自上而下 (top-down) 的贪婪 (greedy) 方法：递归二叉分裂 (recursive binary splitting)。

#### 定义 4.2 (递归二叉分裂)

递归二叉分裂 (Recursive Binary Splitting) 是一种用于构建回归树和分类树的贪婪算法。其基本思想是：

1. 从整个特征空间（即所有训练样本的集合）出发，选择一个特征  $X_j$  及其切分点  $s$ ；
2. 将样本划分为两个矩形区域：

$$R_1(j, s) = \{x \mid x_j \leq s\}, \quad R_2(j, s) = \{x \mid x_j > s\};$$

3. 选择使得残差平方和 (RSS) 最小化的分裂：

$$\min_{j, s} \left\{ \sum_{x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2 \right\},$$

其中  $\hat{y}_{R_m}$  表示区域  $R_m$  内训练样本响应的均值。

4. 在每个新生成的区域内，重复上述步骤，直到满足停止条件（如区域样本数过小或树的深度达到上限）。

该方法称为“递归”，因为分裂过程会不断重复；称为“二叉”，因为每次分裂都将一个区域划分为两个子区域；称为“贪婪”，因为每次分裂只考虑当前最优，而不是全局最优。



#### 东邪的 tip 4.2 (贪婪算法-递归二叉分裂)

贪婪算法 (Greedy Algorithm)：在每一步选择中，都作出在当前看来最优的局部选择，而不从整体上进行考虑。‘自上而下’指的是从树的顶端（在此处，所有观测值都属于同一个空间）。开始依次分裂预测变量空间，每个分裂点都会产生两个新的分支。“贪婪”意味着在建立树的每一步中，最优 (best) 分裂的确定仅限于这一步进程，而不是针对全局去选择那些能够在未来过程中构建出更好树的分裂点。二叉分裂法就是一旦他分割了一次，以后的分割都是在分割好的区域继续分割，不会从两个区域里面个拿出一部分产生一个新的区域即使这样会更好。

#### 命题 4.1 (防止过拟合想要全局最优)

上述方法会在训练集中取得良好的预测效果，却很有可能造成数据的过拟合，导致在测试集上效果不佳。原因在于这种方法产生的树可能过于复杂。一个分裂点更少、规模更小（区域  $R_1, R_2, \dots, R_J$  的个数更少）的树会有更小的方差和更好的可解释性（以增加微小偏差为代价）。

针对上述问题，一种可能的解决办法是：仅当分裂使残差平方和 RSS 的减小量超过某阈值时，才分裂树节点。这种策略能生成较小的树，但可能产生短视的问题：一些起初看起来不值得的分裂在后面可能产生非常好的分裂，也就是说，在下一步中，RSS 大幅减小。

因此，更好的策略是生成一个很大的树  $T_0$ ，然后通过剪枝 (prune) 得到子树 (sub-tree)。

#### 定义 4.3 (代价复杂性剪枝)

代价复杂性剪枝 (Cost Complexity Pruning, 又称最弱链接剪枝 weakest link pruning) 是一种用于从原始大树  $T_0$  中选择最优子树的策略。该方法不是考虑所有可能的子树，而是通过引入调整参数  $\alpha \geq 0$ ，在子树的复杂性和训练误差之间进行权衡。

对于给定的子树  $T$ ，其代价复杂度定义为：

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|,$$

其中  $|T|$  表示子树  $T$  的终端节点数， $R_m$  表示第  $m$  个终端节点对应的区域， $\hat{y}_{R_m}$  是区域  $R_m$  内训练集的平均响应值。

当  $\alpha = 0$  时， $C_\alpha(T)$  仅度量训练误差，最优子树为原始大树  $T_0$ ；随着  $\alpha$  增大，终端节点数较多的子树将因复杂性受到惩罚，使得更小的子树可能成为  $C_\alpha(T)$  的最优解。因此，通过调节  $\alpha$  并最小化  $C_\alpha(T)$ ，我们可以在模型复杂度和泛化能力之间找到平衡，从而选出合适的子树。

也就是最开始的大树太复杂了想砍掉一些枝桠，但是砍的太多就预测的太不准了，砍完枝桠的 RSS 就是  $C_\alpha$  前面的部分。为了鼓励结构尽可能简单，后面加入  $T$  复杂度的权重，权重值为  $\alpha$ 。权重值越高说明越倾向简答的结构。

#### 东邪的 tip 4.3

这是一种二次优化的思想，每当想引入新的维度的指标的时候都可以加入带权重的数。这正是正则化 / 多目标优化的思想。

#### 例题 4.1 算法 8.1 建立回归树

1. 利用递归二叉分裂在训练集中生成一个大树，只有当终端节点包含的观测值个数低于某个最小值时才停止。
2. 对大树进行代价复杂性剪枝，得到一系列最优子树，子树是  $\alpha$  的函数。
3. 利用  $K$  折交叉验证选择  $\alpha$ 。具体做法是将训练集分为  $K$  折。对所有  $k = 1, 2, \dots, K$ ，有：
  - (a) 对训练集上所有不属于第  $k$  折的数据重复步骤 1 和步骤 2，得到与  $\alpha$  一一对应的子树。
  - (b) 求出上述子树在第  $k$  折上的均方预测误差。
 上述操作结束后，每个  $\alpha$  会有相应的  $K$  个均方预测误差，对这  $K$  个值求平均，选出使平均误差最小的  $\alpha$ 。
4. 找出选定的  $\alpha$  值在步骤 2 中对应的子树即可。

#### 定义 4.4 (分类树)

分类树 (classification tree) 和回归树十分相似，它们的区别在于：分类树被用于预测定性变量而非定量变量。也就是研究是什么，而不是是多少的问题。在回归树中，对一给定观测值，响应预测值取它所属的终端节点内训练集的平均响应值。与之相对，对分类树来说，给定观测值被预测为它所属区域内的训练集中最常出现的类 (most commonly occurring class)。

分类树的构造过程和回归树也很像。与回归树一样，分类树采用递归二叉分裂。但在分类树中，RSS 无法作为确定二叉分裂点的准则。一个很自然的替代指标是分类错误率 (classification error rate)。既然要将给定区域内的观测值都分到此区域的训练集上最常出现的类中，那么分类错误率的定义为：此区域的训练集中非最常见

类所占的比例。其表达式如下：

$$E = 1 - \max_k (\hat{p}_{mk}), \quad (8.5)$$

上式中的  $\hat{p}_{mk}$  表示第  $m$  个区域的训练集中的第  $k$  类所占比例。但事实上，分类错误率这一指标在构建树时不够敏感。在实践中，另外两个指标更有优势。

#### 定义 4.5 (基尼系数)

基尼系数 (Gini index) 用于度量分类树节点的纯度，其定义为

$$G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk}), \quad (8.6)$$

其中， $\hat{p}_{mk}$  表示第  $m$  个区域的训练集中第  $k$  类所占的比例。

基尼系数可以理解为  $K$  个类别的总方差。如果所有  $\hat{p}_{mk}$  的取值都接近于 0 或 1，则基尼系数会很小。因而，基尼系数常被视为度量节点纯度 (purity) 的指标：若基尼系数较小，就意味着该节点包含的观测值几乎都来自同一类。也是 Bernoulli 方差

- $p$  表示某个类别的占比；
- $1 - p$  表示“其他类别的占比”；
- $p(1 - p)$  则反映了一个类与其他类的对立程度：当分布均衡时 ( $p$  和  $1 - p$  接近)，该值最大；当一边占绝对优势时，该值趋近于 0。

熵 (Entropy) 也可以用来替代基尼系数作为分类树分裂指标，其定义为

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}, \quad (8.7)$$

其中， $\hat{p}_{mk}$  表示第  $m$  个区域的训练集中第  $k$  类所占的比例。

由于  $0 \leq \hat{p}_{mk} \leq 1$ ，可知  $0 \leq -\hat{p}_{mk} \log \hat{p}_{mk}$ 。显然，如果所有  $\hat{p}_{mk}$  的取值都接近于 0 或 1，则熵值接近于 0。因此，与基尼系数类似，如果一个节点的纯度较高，则熵值较小。事实上，基尼系数和熵在数值上是相当接近的。

在建立分类树的过程中，无论基尼系数还是熵，通常都用于度量某个分裂点的分类质量。与分类错误率相比，这两种方法对节点纯度更敏感。上述三种方法都可以用于树的剪枝，但若希望追求较高的预测准确性，最好选择分类错误率。

## 4.2 装代法

#### 定义 4.6 (装袋法 (Bagging))

在一般情况下难以取得多个训练集，因此可以使用自助法 (Bootstrap) 作为替代，即从某一个单一的训练集中重复取样，生成  $B$  个不同的自助抽样训练集。然后在第  $b$  个自助抽样训练集上拟合模型并得到预测值  $\hat{f}^{*b}(x)$ 。最后对所有预测值取平均，得到装袋法预测函数：

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

决策树 (Decision Tree) 通常具有较高的方差 (high variance)，这意味着如果将同一个训练集随机拆分成两部分，分别建立回归树，结果可能会差别很大。相比之下，像线性回归这类“简单模型”通常方差较小，在不同的数据集上结果差别不会太大。为了降低这种高方差，统计学习里常用一种通用技巧，称为自助法聚集 (bootstrap aggregation)，简称 **bagging**。具体做法是：从训练集里反复有放回地抽样 (bootstrap)，得到多个子集；在每个子集上分别建立一个模型（如决策树）；最后把这些模型的预测结果通过平均或投票的方式组合起来。这样可以显著降低模型的方差，提高泛化性能。装袋法可以显著改善回归模型的预测效果，尤其对决策树十分有效。具体

做法是：利用  $B$  个自助抽样训练集分别构造  $B$  棵回归树，再对其预测结果取平均。这些树通常根深叶茂且未剪枝，每棵树方差高但偏差小，通过平均可有效降低方差。事实证明，成百上千棵树的组合能大幅提升预测精度。

在分类问题中，装袋法同样适用。对于一个给定测试样本，记录  $B$  棵树的预测类别，再采用**多数投票**(majority vote)，即将出现频率最高的类别作为整体预测结果。

#### 定义 4.7 (袋外误差估计)

装袋法可通过袋外 (out-of-bag, OOB) 样本直接估计测试误差，而无需交叉验证。在自助抽样中，每棵树约使用训练样本的三分之二，剩余三分之一即为该树的 OOB 样本。对某个观测值，可用未包含它的约  $B/3$  棵树进行预测，并将结果求平均（回归）或多数投票（分类），得到该观测值的 OOB 预测。综合所有样本即可计算 OOB 均方误差或分类误差，从而得到装袋法测试误差的有效估计。事实证明，当  $B$  足够大时，OOB 误差与留一法交叉验证误差近似等价。



即使装袋法树的集合比单个树解释数据要困难得多，也可以用 RSS（针对装袋法回归树）或基尼系数（针对装袋法分类树），对各预测变量的重要性做出整体概括。

## 4.3 随机森林

#### 定义 4.8 (随机森林)

随机森林 (random forest) 通过对树做去相关处理，实现了对装袋法树的改进。在随机森林中，需要对自助抽样训练集建立一系列决策树，这与装袋法类似。不过，在建立这些决策树时，每个分裂点不再考虑全部的  $p$  个预测变量，而是从中随机抽取  $m$  个预测变量作为候选。该分裂点所用的预测变量只能从这  $m$  个候选变量中选择。在每个分裂点处都要重新抽取  $m$  个预测变量，通常  $m \approx \sqrt{p}$ 。也就是说，每个分裂点所考虑的预测变量个数约等于总预测变量数的平方根。例如，在 Heart 数据集中共有 13 个预测变量，则取  $m = 4$ 。



这个目的就为了防止用装袋法生成的树都长的太相似。比如说都会选择最重要的变量作为顶部分裂点。问题是，与对不相关的量求平均相比，对许多高度相关的量求平均带来的方差减小程度是无法与前者相提并论的。具体而言，在这种情况下，这意味着装袋法与单个树相比不会带来方差的明显降低。但是随机森林因为是随机取出  $m$  个预测变量作为候选，那么最强的变量可能都不在里面，这样的话次强变量等等都可能作为顶部分裂点等塑造出不一样的树。

- 10 个人都参考同一个答案抄作业  $\Rightarrow$  他们错的地方一样，平均结果还是错的。
- 10 个人各自独立思考  $\Rightarrow$  有人错有人对，平均结果更接近真值。

分类聚类然后神经网络把深度学习结合到 Matlab 上边做边学

## 第5章 基础概念

### 定义 5.1 (监督学习)

监督学习 (supervised learning) 是一类利用有标签数据集 (labeled dataset) 训练模型的机器学习方法。数据集表示为  $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ , 其中  $N$  为样本总数,  $\mathbf{x}_i$  为第  $i$  个样本的特征向量 (feature vector),  $y_i$  为对应的标签 (label)。监督学习算法 (supervised learning algorithm) 利用上述数据集生成一个模型, 以样本的特征向量作为输入, 输出用于判断该样本标记的信息。



例如, 一个癌症预测模型可以利用某患者的特征向量, 输出该患者患有癌症的概率。就是每一个数据是知道是癌症或者不是癌症的

### 定义 5.2 (非监督学习)

有别于监督学习, 非监督学习 (unsupervised learning) 只需要包含无标签样本的数据集, 表示为  $\{(\mathbf{x}_i)\}_{i=1}^N$ 。非监督学习算法 (unsupervised learning algorithm) 所产生的模型 (model) 同样接受一个特征向量  $\mathbf{x}$  为输入信息, 并通过数学变换使其变成另外一个对其他任务更有用的向量或数值。



 **笔记** 非监督学习的三类典型任务举例如下:

- 聚类 (clustering): 对电商平台的用户按消费行为自动分组, 模型输出每个用户所属的类簇标记, 无需事先告知“应分几类”。
- 降维 (dimensionality reduction): 将每位用户由上千维的行为特征向量压缩为二维向量, 便于可视化分析用户分布。
- 异常值检测 (outlier detection): 对每笔信用卡交易输出一个实数值, 该值越大表示该笔交易与“正常”交易差异越大, 从而识别潜在的欺诈行为。

### 定义 5.3 (半监督学习)

半监督学习 (semi-supervised learning) 可利用掺杂着有标签和无标签样本的数据集进行学习, 通常情况下无标签样本的数量远超过有标签样本。半监督学习算法 (semi-supervised learning algorithm) 的功能和原理与监督学习算法大同小异, 区别在于算法可以利用大量无标注的样本学得更好的模型。



### 定义 5.4 (强化学习)

在强化学习 (reinforcement learning) 中, 我们假设机器“生活”在一个环境中, 并可以感知当前环境的状态 (state), 状态表示为特征向量。在每个状态下, 机器可以通过执行不同动作而获取相应的奖励, 同时进入另一个环境状态。强化学习的目的是学习选择行动的策略 (policy), 使得机器在长期交互过程中累计获得的奖励最大化。



 **笔记** 强化学习的典型应用场景举例如下:

- 游戏对战: AlphaGo 以棋盘局面作为状态, 以落子位置作为动作, 通过不断与自身对弈获取胜负奖励, 最终学得超越人类的博弈策略。
- 自动驾驶: 车辆以道路环境 (车速、障碍物位置等) 作为状态, 以加速、制动、转向作为动作, 通过安全行驶里程作为奖励信号, 学习最优驾驶策略。
- 推荐系统: 平台以用户历史行为作为状态, 以推送内容作为动作, 以用户点击或停留时长作为奖励, 持续优化内容推荐策略。

 **笔记** 分类 (classification) 与回归 (regression) 的核心区别在于输出值的类型:

- 分类: 输出为离散值, 即样本所属的类别标签。例如, 判断一封邮件是否为垃圾邮件 (是/否), 或识别手

写数字属于 0 到 9 中的哪一类。

- 回归：输出为连续实数值。例如，根据房屋面积、地段等特征向量预测房价，或根据历史数据预测某股票次日的收盘价格。

 **笔记** 浅度学习 (shallow learning) 与深度学习 (deep learning) 的区别主要体现在模型结构的复杂程度：

- 浅度学习：模型结构较简单，通常只有一层或少数几层变换，例如逻辑回归、支持向量机 (SVM)、决策树等。此类模型对特征工程依赖较强，需要人工设计输入特征。
- 深度学习：模型由多层非线性变换堆叠而成 (即深层神经网络)，能够自动从原始数据中逐层提取抽象特征，无需大量人工特征设计。典型应用包括图像识别、语音识别和自然语言处理。

一般而言，深度学习在数据量充足时表现更优，而浅度学习在小样本或可解释性要求较高的场景下仍具有重要价值。

#### 东邪的 tip 5.1

选方差当惩罚函数一个是因为绝对值函数不平滑，一个是可以惩罚偏差

#### 定义 5.5 (逻辑回归——解决把结果映射到 01 之间以及把向量乘成数字)

逻辑回归 (logistic regression) 是一种用于二分类任务的监督学习算法。给定样本的特征向量  $\mathbf{x} \in \mathbb{R}^n$ ，逻辑回归通过 **sigmoid** 函数将线性组合  $\mathbf{w}^\top \mathbf{x} + b$  映射到区间  $(0, 1)$ ，输出样本属于正类的概率：

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

其中  $\mathbf{w} \in \mathbb{R}^n$  为权重向量， $b \in \mathbb{R}$  为偏置项。模型参数通常通过最大化对数似然函数 (即最小化交叉熵损失) 估计得到。预测时，以  $\hat{y} \geq 0.5$  判为正类，否则判为负类。



 **笔记** 他用最大似然来衡量

## 第 6 章 《Attention Is All You Need》中文版

### 6.1 摘要

目前主流的序列转换模型建立在复杂的循环神经网络或卷积神经网络之上，通常包含一个编码器和一个解码器。性能最优的模型还通过注意力机制将编码器与解码器连接起来。本文提出一种全新的简单网络架构——Transformer，它完全基于注意力机制，彻底摒弃了循环结构与卷积操作。在两项机器翻译任务上的实验表明，该模型在翻译质量上更优，同时具备更强的并行性，所需训练时间也大幅缩短。在 WMT 2014 英德翻译任务上，我们的模型取得了 28.4 的 BLEU 分数，比现有最优结果（包括集成模型）提升了 2 个 BLEU 以上。在 WMT 2014 英法翻译任务上，我们的模型仅用 8 块 GPU 训练 3.5 天，便以单模型创下 41.8 的 BLEU 新纪录，训练成本仅为文献中最优模型的一小部分。此外，我们将 Transformer 成功应用于英语成分句法分析任务（无论训练数据规模大小），证明其具备良好的泛化能力。

### 6.2 引言

循环神经网络，尤其是长短期记忆网络(LSTM)[hochreiter1997long]和门控循环神经网络[cho2014learning]，已在序列建模与转换问题（如语言建模和机器翻译）中牢固确立了其最先进方法的地位[sutskever2014sequence, cho2014learning, bahdanau2014neural]。此后，大量研究持续推动着循环语言模型与编码器-解码器架构的边界[wu2016google, luong2015effective, jozefowicz2016exploring]。

循环模型通常沿输入和输出序列的符号位置逐步推进计算。将位置与计算时间步对齐后，模型以上一时刻的隐藏状态  $h_{t-1}$  和当前位置  $t$  的输入为函数，生成一系列隐藏状态  $h_t$ 。这种本质上的顺序性使得训练样本内部无法并行化，在序列较长时尤为制约性能，因为内存限制会压缩跨样本的批处理规模。近期工作通过分解技巧[kuchaiev2017factorization]和条件计算[shazeer2017outrageously]在计算效率上取得了显著提升，后者同时也改善了模型性能。然而，顺序计算这一根本约束依然存在。

注意力机制已成为各类序列建模与转换模型中不可或缺的组成部分，它允许建模输入或输出序列中任意距离的依赖关系[bahdanau2014neural, structuredattentionnetworks]。然而，除少数例外[decomposableattentionmodel]，注意力机制几乎都与循环网络配合使用。

本文提出 Transformer——一种完全摒弃循环结构、仅依赖注意力机制来捕捉输入与输出之间全局依赖关系的模型架构。Transformer 具备更强的并行化能力，仅需在 8 块 P100 GPU 上训练短短十二小时，便可在翻译质量上达到新的最先进水平。

### 6.3 背景

减少顺序计算这一目标，同样是 Extended Neural GPU[extendedneuralgpu]、ByteNet[kalchbrenner2016neural]和 ConvS2S[gehring2017convolutional]的出发点。这些模型均以卷积神经网络为基本构件，能够并行计算所有输入和输出位置的隐藏表示。但在这些模型中，关联两个任意输入或输出位置信号所需的操作次数，随位置间距的增大而增长——ConvS2S 呈线性增长，ByteNet 呈对数增长。这使得学习远距离位置之间的依赖关系更加困难[hochreiter2001gradient]。在 Transformer 中，这一代价被降低为常数次操作，尽管代价是因对注意力权重位置取平均而导致有效分辨率下降——我们通过第 3.2 节介绍的多头注意力机制来抵消这一影响。

自注意力（有时称为内部注意力）是一种将单个序列中不同位置相互关联、从而计算序列表示的注意力机制。自注意力已在阅读理解、抽象摘要、文本蕴含及与任务无关的句子表示学习等多种任务中取得成功[cheng2016long, decomposableattentionmodel, paulus2017deep, lin2017structured]。

端到端记忆网络基于循环注意力机制（而非序列对齐的循环结构），已被证明在简单语言问答和语言建模任务上表现良好 [sukhbaatar2015]。

然而，据我们所知，Transformer 是首个完全依赖自注意力来计算输入和输出表示、不使用任何序列对齐 RNN 或卷积的转换模型。在接下来的章节中，我们将描述 Transformer，阐述自注意力的动机，并讨论其相较于 [sutskever2014sequence, bahdanau2014neural] 等模型的优势。

## 6.4 模型架构

大多数具有竞争力的神经序列转换模型都采用编码器-解码器结构 [cho2014learning, bahdanau2014neural, sutskever2014sequence]。编码器将输入符号表示序列  $(x_1, \dots, x_n)$  映射为连续表示序列  $\mathbf{z} = (z_1, \dots, z_n)$ 。给定  $\mathbf{z}$  后，解码器逐步生成输出符号序列  $(y_1, \dots, y_m)$ ，每次生成一个元素。在每一步中，模型是自回归的 [graves2013generating]，即在生成下一个符号时，将已生成的符号作为额外输入。

Transformer 遵循上述整体架构，对编码器和解码器均采用堆叠的自注意力层与逐点全连接层，分别对应图 1 的左半部分和右半部分。

### 6.4.1 编码器与解码器堆栈

**编码器：**编码器由  $N = 6$  个相同层堆叠而成。每层包含两个子层：第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。在每个子层周围采用残差连接 [he2016deep]，随后进行层归一化 [ba2016layer]。即每个子层的输出为  $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中  $\text{Sublayer}(x)$  为该子层自身实现的函数。为便于使用残差连接，模型中所有子层及嵌入层的输出维度均为  $d_{\text{model}} = 512$ 。

**解码器：**解码器同样由  $N = 6$  个相同层堆叠而成。在编码器每层的两个子层基础上，解码器额外插入第三个子层，对编码器堆栈的输出执行多头注意力。与编码器类似，每个子层周围同样采用残差连接并进行层归一化。此外，对解码器堆栈中的自注意力子层进行了修改，通过掩码 (Masking) 防止当前位置关注到后续位置。这种掩码结合输出嵌入偏移一个位置的设计，确保位置  $i$  的预测只能依赖位置  $i$  之前的已知输出。

### 6.4.2 注意力

注意力函数可描述为：将一个查询 (query) 和一组键值对 (key-value pairs) 映射为输出，其中查询、键、值和输出均为向量。输出是值的加权求和，每个值的权重由查询与对应键的相容性函数计算得到。

#### 6.4.2.1 缩放点积注意力

我们将本文的注意力称为“缩放点积注意力”（图 2）。输入包含维度为  $d_k$  的查询和键，以及维度为  $d_v$  的值。将查询与所有键做点积，除以  $\sqrt{d_k}$  后，应用 softmax 函数得到各值的权重。

在实践中，我们将一组查询打包成矩阵  $Q$ ，同时将键和值分别打包成矩阵  $K$  和  $V$ ，并行计算注意力函数，输出矩阵为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

最常用的两种注意力函数是加性注意力 [bahdanau2014neural] 和点积（乘法）注意力。点积注意力与本文算法完全相同，区别仅在于缩放因子  $\frac{1}{\sqrt{d_k}}$ 。加性注意力使用单隐层前馈网络计算相容性函数。二者理论复杂度相近，但点积注意力在实践中速度更快、空间效率更高，因为可以借助高度优化的矩阵乘法代码实现。

当  $d_k$  较小时，两种机制性能相当；当  $d_k$  较大时，不加缩放的点积注意力性能逊于加性注意力 [DBLP:journals/corr/BritzGL16]。我们推测，当  $d_k$  较大时，点积结果幅值增大，将 softmax 函数推入梯度极小的区域。为抵消这一效应，我们将点积结果除以  $\frac{1}{\sqrt{d_k}}$  进行缩放。

### 6.4.2.2 多头注意力

我们发现，与其用  $d_{\text{model}}$  维的键、值和查询执行单次注意力函数，不如将查询、键和值分别经过  $h$  次不同的可学习线性投影，投影到  $d_k$ 、 $d_k$  和  $d_v$  维度，效果更佳。在每组投影后的查询、键和值上并行执行注意力函数，得到  $d_v$  维输出值，将其拼接后再次投影，得到最终输出，如图 2 所示。

多头注意力使模型能够在不同位置上同时关注来自不同表示子空间的信息，而单头注意力的平均化操作会抑制这一能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

其中  $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

投影矩阵参数为  $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ ,  $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ ,  $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ ,  $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

本文采用  $h = 8$  个并行注意力头，每个头的维度为  $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度缩减，总计算量与全维度单头注意力相近。

### 6.4.2.3 注意力在模型中的应用

Transformer 以三种不同方式使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自上一解码器层，键和值来自编码器的输出。这使解码器的每个位置都能关注输入序列的所有位置，与 seq2seq 模型中的典型编码器-解码器注意力机制 [wu2016google, bahdanau2014neural, decomposableattentionmodel] 类似。
- 编码器包含自注意力层。在自注意力层中，键、值和查询均来自同一处——上一编码器层的输出。编码器中每个位置都可以关注编码器上一层的所有位置。
- 同样，解码器中的自注意力层允许每个位置关注解码器中该位置及之前的所有位置。为保持自回归性质，需要阻止解码器中信息向左流动。我们在缩放点积注意力内部实现这一点：将 softmax 输入中对应非法连接的所有值掩码为  $-\infty$ 。

### 6.4.3 逐位置前馈网络

除注意力子层外，编码器和解码器的每一层还包含一个全连接前馈网络，对每个位置分别且相同地应用。该网络由两个线性变换组成，中间加入 ReLU 激活函数：

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

不同位置的线性变换相同，但不同层之间使用不同参数。也可将其理解为两个核大小为 1 的卷积。输入和输出维度为  $d_{\text{model}} = 512$ ，内层维度为  $d_{\text{ff}} = 2048$ 。

### 6.4.4 嵌入与 Softmax

与其他序列转换模型类似，我们使用可学习的嵌入将输入词元和输出词元转换为  $d_{\text{model}}$  维向量。同时使用常规的可学习线性变换和 softmax 函数，将解码器输出转换为预测下一词元的概率。本模型在两个嵌入层与 pre-softmax 线性变换之间共享权重矩阵 [press2016using]，在嵌入层中将权重乘以  $\sqrt{d_{\text{model}}}$ 。

### 6.4.5 位置编码

由于本模型不含循环结构也不含卷积，为使模型能够利用序列的顺序信息，必须向模型注入词元在序列中的相对或绝对位置信息。为此，我们在编码器和解码器堆栈底部的输入嵌入上叠加“位置编码”。位置编码与嵌入的维度相同，均为  $d_{\text{model}}$ ，因此二者可以直接相加。位置编码有多种选择，包括可学习的和固定的 [gehring2017convolutional]。

本文使用不同频率的正弦和余弦函数：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

其中  $pos$  为位置,  $i$  为维度索引。每个维度的位置编码对应一条正弦曲线, 波长从  $2\pi$  到  $10000 \cdot 2\pi$  构成等比数列。选择该函数是因为我们假设它能使模型轻松学会通过相对位置进行注意力计算——对于任意固定偏移  $k$ ,  $PE_{pos+k}$  可表示为  $PE_{pos}$  的线性函数。

我们还实验了使用可学习的位置嵌入 [gehring2017convolutional], 发现两种方式结果几乎相同 (见表 3 第 (E) 行)。最终选择正弦版本, 因为它可能允许模型外推至训练时未见过的更长序列。

## 6.5 为什么使用自注意力

本节将自注意力层与循环层、卷积层在多个方面进行比较。这些层通常用于将一段变长符号表示序列  $(x_1, \dots, x_n)$  映射为另一段等长序列  $(z_1, \dots, z_n)$ , 其中  $x_i, z_i \in \mathbb{R}^d$ , 例如典型序列转换编码器或解码器中的隐藏层。我们从三个方面论证使用自注意力的动机。

其一是每层的总计算复杂度。其二是可并行化的计算量, 以所需最少顺序操作数衡量。

其三是网络中长距离依赖之间的路径长度。学习长距离依赖是许多序列转换任务的核心挑战。影响这一能力的关键因素之一, 是前向与反向信号在网络中需要经过的路径长度。输入与输出序列中任意位置组合之间的路径越短, 越容易学习长距离依赖 [hochreiter2001gradient]。因此, 我们也比较了由不同类型层构成的网络中, 任意两个输入输出位置之间的最大路径长度。

如表 1 所示, 自注意力层以常数次顺序操作连接所有位置, 而循环层需要  $O(n)$  次顺序操作。在计算复杂度方面, 当序列长度  $n$  小于表示维度  $d$  时, 自注意力层比循环层更快——这在机器翻译中最先进模型所使用的句子表示 (如词片 [wu2016google] 和字节对 [sennrich2015neural] 表示) 中最为常见。

为提升处理超长序列任务时的计算性能, 自注意力可限制为只关注以各输出位置为中心、大小为  $r$  的邻域。这将使最大路径长度增至  $O(n/r)$ , 我们计划在未来工作中进一步研究这一方法。

核宽为  $k < n$  的单层卷积层无法连接所有输入输出位置对。若要做到这一点, 连续核情况下需要堆叠  $O(n/k)$  层卷积, 膨胀卷积 [yu2015multi] 情况下需要  $O(\log_k(n))$  层, 这都会增加网络中任意两个位置之间最长路径的长度。卷积层的计算代价通常比循环层高出  $k$  倍。然而, 可分离卷积 [chollet2016xception] 可将复杂度大幅降低至  $O(k \cdot n \cdot d + n \cdot d^2)$ 。即便取  $k = n$ , 可分离卷积的复杂度也与自注意力层加逐点前馈层的组合相当——正是本文模型所采用的方案。

自注意力还有一个附带好处: 可产生更具可解释性的模型。我们对模型的注意力分布进行了检视, 并在附录中列举和讨论了若干示例。不仅各个注意力头明显学会了执行不同的任务, 许多头还表现出与句子句法和语义结构相关的行为。

## 6.6 训练

本节介绍模型的训练方案。

### 6.6.1 训练数据与批处理

我们在标准的 WMT 2014 英德数据集上进行训练, 该数据集约包含 450 万个句子对。句子采用字节对编码 [sennrich2015neural], 源语言与目标语言共享约 37000 个词元的词表。对于英法翻译, 我们使用规模更大的 WMT 2014 英法数据集, 共 3600 万个句子, 词元切分为 32000 个词片词表 [wu2016google]。句子对按近似序列长度打包成批次, 每个训练批次约包含 25000 个源词元和 25000 个目标词元。

### 6.6.2 硬件与训练计划

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。使用本文所描述超参数的基础模型, 每个训练步耗时约 0.4 秒, 共训练 100000 步, 历时约 12 小时。大模型每步耗时 1.0 秒, 共训练 300000 步, 历时 3.5 天。

### 6.6.3 优化器

我们使用 Adam 优化器 [kingma2014adam]，参数设置为  $\beta_1 = 0.9$ ， $\beta_2 = 0.98$ ， $\epsilon = 10^{-9}$ 。训练过程中按以下公式动态调整学习率：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5})$$

即在前  $warmup\_steps$  步线性增大学习率，之后按步数的平方根倒数衰减。本文设  $warmup\_steps = 4000$ 。

### 6.6.4 正则化

训练中采用三种正则化方式：

**残差 Dropout**。在每个子层的输出加到子层输入并归一化之前，对其施加 Dropout [srivastava2014dropout]。此外，在编码器和解码器堆栈中，对嵌入与位置编码之和也施加 Dropout。基础模型的丢弃率为  $P_{drop} = 0.1$ 。

**标签平滑**。训练时采用值为  $\epsilon_{ls} = 0.1$  的标签平滑 [szegedy2015rethinking]。这会使模型变得更加“不确定”，从而损害困惑度，但能提升准确率和 BLEU 分数。

## 6.7 实验结果

### 6.7.1 机器翻译

在 WMT 2014 英德翻译任务上，大型 Transformer 模型（表 2 中的 Transformer (big)）以超过 2.0 BLEU 的优势超越了此前所有已报告的最优模型（包括集成模型），以 28.4 的 BLEU 分数创下新的最先进水平。该模型的配置见表 3 最后一行，在 8 块 P100 GPU 上训练耗时 3.5 天。即便是我们的基础模型，也以远低于任何竞争模型的训练成本超越了此前所有已发表的模型和集成模型。

在 WMT 2014 英法翻译任务上，我们的大模型取得了 41.0 的 BLEU 分数，超越了此前所有已发表的单模型，训练成本不足此前最优模型的四分之一。英法任务的大型 Transformer 模型使用的 Dropout 率为  $P_{drop} = 0.1$ ，而非 0.3。

对于基础模型，我们对最后 5 个检查点（每 10 分钟保存一次）取平均，得到单一模型。对于大模型，则对最后 20 个检查点取平均。推理时使用束搜索，束大小为 4，长度惩罚系数  $\alpha = 0.6$  [wu2016google]，这些超参数均经过开发集实验选定。推理时最大输出长度设为输入长度加 50，但在可能时提前终止 [wu2016google]。

表 2 汇总了我们的实验结果，并将翻译质量和训练成本与文献中其他模型架构进行了比较。训练模型所用的浮点运算次数，由训练时间、所用 GPU 数量以及每块 GPU 的单精度浮点运算峰值估算得出。

### 6.7.2 模型变体

为评估 Transformer 各组件的重要性，我们以不同方式对基础模型进行变体实验，在开发集 newstest2013 上衡量英德翻译性能的变化，结果见表 3。推理使用上节所述的束搜索，但不进行检查点平均。

在表 3 第 (A) 组，我们按第 3.2.2 节的描述，在保持计算量不变的前提下，改变注意力头数及注意力键值维度。结果显示，单头注意力比最优设置差 0.9 BLEU，但头数过多时质量同样下降。

在第 (B) 组，我们观察到减小注意力键值维度  $d_k$  会损害模型质量，这表明判断相容性并不容易，比点积更复杂的相容性函数或许会更有益。在第 (C) 和 (D) 组，进一步验证了更大的模型效果更好，Dropout 对于防止过拟合非常有效。在第 (E) 行，用可学习的位置嵌入 [gehring2017convolutional] 替换正弦位置编码后，结果与基础模型几乎相同。

### 6.7.3 英语成分句法分析

为评估 Transformer 能否泛化到其他任务，我们在英语成分句法分析任务上进行了实验。该任务有其特殊挑战：输出受到强结构约束，且通常远长于输入；此外，在小数据场景下，RNN seq2seq 模型难以达到最先进水平 [vinyals2015grammar]。

我们在 Penn Treebank [marcus1993building] 的华尔街日报 (WSJ) 部分 (约 4 万训练句) 上训练了一个 4 层、 $d_{\text{model}} = 1024$  的 Transformer。同时还在半监督设置下进行了训练，利用了约 1700 万句的高置信度语料和 BerkeleyParser 语料 [vinyals2015grammar, zhu2013fast]。纯 WSJ 设置使用 16K 词元词表，半监督设置使用 32K 词元词表。

我们仅在第 22 节开发集上进行了少量实验以选择 Dropout 率 (注意力和残差，见第 5.4 节)、学习率和束大小，其余所有参数均沿用英德基础翻译模型的设置。推理时最大输出长度设为输入长度加 300，束大小为 21， $\alpha = 0.3$  (纯 WSJ 和半监督设置相同)。

表 4 的结果表明，尽管缺乏针对特定任务的调优，我们的模型表现出乎意料地好，优于此前所有已报告模型，仅次于循环神经网络文法模型 (RNNG) [dyer2016recurrent]。与 RNN seq2seq 模型 [vinyals2015grammar] 不同，即使仅在 4 万句的 WSJ 训练集上训练，Transformer 也超越了 BerkeleyParser [petrov2006learning]。

## 6.8 结论

本文提出了 Transformer——首个完全基于注意力机制的序列转换模型，以多头自注意力取代了编码器-解码器架构中最常用的循环层。

在翻译任务上，Transformer 的训练速度显著快于基于循环或卷积层的架构。在 WMT 2014 英德和英法翻译任务上均取得了新的最先进水平，前者甚至超越了此前所有已报告的集成模型。

我们对基于注意力模型的未来充满期待，并计划将其应用于其他任务。我们还计划将 Transformer 扩展到文本以外的输入输出模态，并研究局部受限注意力机制，以高效处理图像、音频和视频等大规模输入输出。减少生成过程的顺序性也是我们的研究目标之一。

本文训练和评估模型所用的代码已开源，地址为 <https://github.com/tensorflow/tensor2tensor>。

## 第 7 章 Attention Is All You Need

### 7.1 Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

### 7.2 Introduction

Recurrent neural networks, long short-term memory [hochreiter1997long] and gated recurrent [cho2014learning] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [sutskever2014sequence, cho2014learning, bahdanau2014neural]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [wu2016google, luong2015effective, jozefowicz2016exploring].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states  $h_t$ , as a function of the previous hidden state  $h_{t-1}$  and the input for position  $t$ . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [kuchaiev2017factorization] and conditional computation [shazeer2017outrageously], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [bahdanau2014neural, structuredattentionnetworks]. In all but a few cases [decomposableattentionmodel], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

### 7.3 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [extendedneuralgpu], ByteNet [kalchbrenner2016neural] and ConvS2S [gehring2017convolutional], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these

models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [hochreiter2001gradient]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in Section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [cheng2016long, decomposableattentionmodel, paulus2017deep, lin2017structured].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [sukhbaatar2015].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [sutskever2014sequence, bahdanau2014neural] and [recurrentleastsquares, lecun2015deep].

## 7.4 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [cho2014learning, bahdanau2014neural, sutskever2014sequence]. Here, the encoder maps an input sequence of symbol representations  $(x_1, \dots, x_n)$  to a sequence of continuous representations  $\mathbf{z} = (z_1, \dots, z_n)$ . Given  $\mathbf{z}$ , the decoder then generates an output sequence  $(y_1, \dots, y_m)$  of symbols one element at a time. At each step the model is auto-regressive [graves2013generating], consuming the previously generated symbols as additional input when generating the next.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

### 7.4.1 Encoder and Decoder Stacks

**Encoder:** The encoder is composed of a stack of  $N = 6$  identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [he2016deep] around each of the two sub-layers, followed by layer normalization [ba2016layer]. That is, the output of each sub-layer is  $\text{LayerNorm}(x + \text{Sublayer}(x))$ , where  $\text{Sublayer}(x)$  is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension  $d_{\text{model}} = 512$ .

**Decoder:** The decoder is also composed of a stack of  $N = 6$  identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position  $i$  can depend only on the known outputs at positions less than  $i$ .

### 7.4.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

### 7.4.2.1 Scaled Dot-Product Attention

We call our particular attention “Scaled Dot-Product Attention” (Figure 2). The input consists of queries and keys of dimension  $d_k$ , and values of dimension  $d_v$ . We compute the dot products of the query with all keys, divide each by  $\sqrt{d_k}$ , and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix  $Q$ . The keys and values are also packed together into matrices  $K$  and  $V$ . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The two most commonly used attention functions are additive attention [bahdanau2014neural], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of  $\frac{1}{\sqrt{d_k}}$ . Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of  $d_k$  the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of  $d_k$  [DBLP:journals/corr/BritzGLL17]. We suspect that for large values of  $d_k$ , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by  $\frac{1}{\sqrt{d_k}}$ .

### 7.4.2.2 Multi-Head Attention

Instead of performing a single attention function with  $d_{\text{model}}$ -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values  $h$  times with different, learned linear projections to  $d_k$ ,  $d_k$  and  $d_v$  dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding  $d_v$ -dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where  $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Where the projections are parameter matrices  $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ ,  $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ ,  $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$  and  $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ .

In this work we employ  $h = 8$  parallel attention layers, or heads. For each of these we use  $d_k = d_v = d_{\text{model}}/h = 64$ . Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

### 7.4.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In “encoder-decoder attention” layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [wu2016google, bahdanau2014neural, decomposableattentionmodel].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to  $-\infty$ ) all values in the input to the softmax which correspond to illegal connections.

### 7.4.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is  $d_{\text{model}} = 512$ , and the inner-layer has dimensionality  $d_{\text{ff}} = 2048$ .

### 7.4.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension  $d_{\text{model}}$ . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [press2016using]. In the embedding layers, we multiply those weights by  $\sqrt{d_{\text{model}}}$ .

### 7.4.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add “positional encodings” to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension  $d_{\text{model}}$  as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [gehring2017convolutional].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

where  $pos$  is the position and  $i$  is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from  $2\pi$  to  $10000 \cdot 2\pi$ . We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset  $k$ ,  $PE_{pos+k}$  can be represented as a linear function of  $PE_{pos}$ .

We also experimented with using learned positional embeddings [gehring2017convolutional] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

## 7.5 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations  $(x_1, \dots, x_n)$  to another sequence of equal length

$(z_1, \dots, z_n)$ , with  $x_i, z_i \in \mathbb{R}^d$ , such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [hochreiter2001gradient]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires  $O(n)$  sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length  $n$  is smaller than the representation dimensionality  $d$ , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [wu2016google] and byte-pair [sennrich2015neural] representations.

To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size  $r$  in the input sequence centered around the respective output position. This would increase the maximum path length to  $O(n/r)$ . We plan to investigate this approach further in future work.

A single convolutional layer with kernel width  $k < n$  does not connect all pairs of input and output positions. Doing so requires a stack of  $O(n/k)$  convolutional layers in the case of contiguous kernels, or  $O(\log_k(n))$  in the case of dilated convolutions [yu2015multi], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of  $k$ . Separable convolutions [chollet2016xception], however, decrease the complexity considerably, to  $O(k \cdot n \cdot d + n \cdot d^2)$ . Even with  $k = n$ , however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

## 7.6 Training

This section describes the training regime for our models.

### 7.6.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [sennrich2015neural], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [wu2016google]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

### 7.6.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000

steps or 12 hours. For our big models, step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

### 7.6.3 Optimizer

We used the Adam optimizer [kingma2014adam] with  $\beta_1 = 0.9$ ,  $\beta_2 = 0.98$  and  $\epsilon = 10^{-9}$ . We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5})$$

This corresponds to increasing the learning rate linearly for the first *warmup\_steps* training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used *warmup\_steps* = 4000.

### 7.6.4 Regularization

We employ three types of regularization during training:

**Residual Dropout.** We apply dropout [srivastava2014dropout] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of  $P_{drop} = 0.1$ .

**Label Smoothing.** During training, we employed label smoothing of value  $\epsilon_{ls} = 0.1$  [szegedy2015rethinking]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

## 7.7 Results

### 7.7.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models including ensembles by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate  $P_{drop} = 0.1$ , instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty  $\alpha = 0.6$  [wu2016google]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length +50, but terminate early when possible [wu2016google].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU.

### 7.7.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

In Table 3 rows (B), we observe that reducing the attention key size  $d_k$  hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [gehring2017convolutional], and observe nearly identical results to the base model.

### 7.7.3 English Constituency Parsing

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [vinyals2015grammar].

We trained a 4-layer transformer with  $d_{\text{model}} = 1024$  on the Wall Street Journal (WSJ) portion of the Penn Treebank [marcus1993building], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkeleyParser corpora from with approximately 17M sentences [vinyals2015grammar, zhu2013fast]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we increased the maximum output length to input length +300. We used a beam size of 21 and  $\alpha = 0.3$  for both WSJ only and the semi-supervised setting.

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [dyer2016recurrent].

In contrast to RNN sequence-to-sequence models [vinyals2015grammar], the Transformer outperforms the BerkeleyParser [petrov2006learning] even when training only on the WSJ training set of 40K sentences.

## 7.8 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. We achieved a new state of the art on both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goal of ours.

The code we used to train and evaluate our models is available at <https://github.com/tensorflow/tensor2tensor>.

## 第 8 章 《Deep Neural Networks for YouTube Recommendations》中文版

### 8.1 摘要

YouTube 是现存规模最大、最复杂的工业级推荐系统之一。本文从高层视角描述该系统，重点介绍深度学习所带来的显著性能提升。全文按照经典的两阶段信息检索范式组织：首先详细介绍一个深度候选生成模型，然后描述一个独立的深度排序模型。我们还将分享在设计、迭代和维护这一对用户产生巨大影响的大规模推荐系统过程中所积累的实用经验与洞见。

**关键词：**推荐系统；深度学习；可扩展性

### 8.2 引言

YouTube 是全球最大的视频创作、分享与发现平台。YouTube 推荐系统负责帮助超过十亿用户从持续增长的视频库中发现个性化内容。本文重点关注深度学习近期对 YouTube 视频推荐系统所产生的巨大影响。图 1 展示了 YouTube 移动应用首页上的推荐界面。

YouTube 视频推荐面临以下三大主要挑战：

- **规模 (Scale)：**许多在小规模问题上表现良好的推荐算法在我们的规模下无法正常工作。处理 YouTube 庞大的用户群体和内容库，需要高度专业化的分布式学习算法和高效的服务系统。
- **时效性 (Freshness)：**YouTube 的内容库极为动态，每秒都有大量视频上传。推荐系统需要足够灵敏，既能为新上传的内容建模，又能捕捉用户的最新行为。在探索/利用 (exploration/exploitation) 视角下，新内容与成熟视频之间的平衡是一个核心问题。
- **噪声 (Noise)：**YouTube 用户的历史行为由于稀疏性及各种不可观测的外部因素，天然难以预测。我们几乎无法获得用户满意度的真实标签，只能对含噪的隐式反馈信号建模。此外，内容相关的元数据结构混乱，缺乏明确的本体定义。算法需要对训练数据的这些特性保持鲁棒性。

与谷歌旗下其他产品领域协同，YouTube 已完成向深度学习的根本性范式转变，将其作为几乎所有学习问题的通用解决方案。我们的系统构建在 Google Brain 之上，该平台近期以 TensorFlow 的形式开源。TensorFlow 提供了一个灵活的框架，可在大规模分布式训练环境下实验各种深度神经网络架构。我们的模型学习约十亿个参数，并在数千亿个样本上进行训练。

与大量矩阵分解方法的研究相比，将深度神经网络应用于推荐系统的工作相对较少。神经网络已被用于新闻推荐、引文推荐和评分预测；协同过滤被构建为深度神经网络，自编码器亦被用于此目的。Elkahky 等人利用深度学习进行跨域用户建模；在基于内容的场景中，Borges 等人将深度神经网络用于音乐推荐。

本文的组织结构如下：第 2 节简要介绍系统概述；第 3 节更详细地描述候选生成模型，包括其训练方式以及如何用于服务推荐，实验结果将展示该模型如何从更深的隐藏层和额外的异质信号中受益；第 4 节详细介绍排序模型，包括如何改进经典逻辑回归以训练预测期望观看时长（而非点击概率）的模型，实验结果将表明隐藏层深度在此同样有益；最后，第 5 节呈现我们的结论与经验教训。

### 8.3 系统概述

我们推荐系统的整体架构如图 2 所示。该系统由两个神经网络组成：一个用于候选生成 (candidate generation)，另一个用于排序 (ranking)。

候选生成网络以用户在 YouTube 上的活动历史作为输入，从庞大的视频库中检索出一小部分（数百个）可能与用户相关的视频。这些候选视频具有较高的精确率，能够总体上满足用户的相关性需求。候选生成网络仅

通过协同过滤提供粗粒度的个性化。用户之间的相似性通过粗粒度特征来表达，例如观看视频的 ID、搜索词令牌以及人口统计信息。

在列表中呈现少量“最佳”推荐，需要对候选视频的相对重要性进行细粒度的区分，以实现高召回率。排序网络通过使用描述视频与用户的丰富特征集，依据目标函数为每个视频分配一个分数来完成这一任务。得分最高的视频按分数排序后呈现给用户。

两阶段推荐方法使我们能够从数百万规模的庞大视频库中生成推荐，同时确保最终呈现在用户设备上的少量视频是个性化的、能够吸引用户的。此外，这一设计还支持融合来自其他来源的候选视频，例如早期工作中描述的那些来源。

在开发过程中，我们大量使用离线指标（精确率、召回率、排序损失等）来指导系统的迭代改进。然而，对于算法或模型有效性的最终判断，我们依赖于在线 A/B 实验。在线实验可以衡量点击率、观看时长以及其他用户参与度指标的细微变化。这一点至关重要，因为在线 A/B 实验结果并不总是与离线实验结果相关。

## 8.4 候选生成

在候选生成阶段，庞大的 YouTube 视频库被精简为数百个可能与用户相关的视频。本文所描述的推荐系统的前身是一种在排序损失下训练的矩阵分解方法。我们神经网络模型的早期迭代版本通过仅嵌入用户历史观看记录的浅层网络来模拟这种分解行为。从这个角度来看，我们的方法可以视为分解技术的非线性推广。

### 8.4.1 将推荐建模为分类

我们将推荐问题建模为极端多分类问题，预测任务转化为：给定用户  $U$  和上下文  $C$ ，在包含数百万视频  $i$ （类别）的视频库  $V$  中，精确分类用户在时刻  $t$  将观看的特定视频  $w_t$ ，

$$P(w_t = i | U, C) = \frac{e^{v_i^u}}{\sum_{j \in V} e^{v_j^u}}$$

其中  $u \in \mathbb{R}^N$  表示用户与上下文对的高维“嵌入（embedding）”， $v_j \in \mathbb{R}^N$  表示各候选视频的嵌入。在此设定下，嵌入即是稀疏实体（单个视频、用户等）映射到  $\mathbb{R}^N$  中稠密向量的映射。深度神经网络的任务是以用户的历史记录和上下文为输入，学习用户嵌入  $u$ ，使其能够通过 softmax 分类器对视频进行有效区分。

尽管 YouTube 上存在显式反馈机制（点赞/点踩、产品内调查等），我们仍选择使用观看行为这一隐式反馈来训练模型，以用户看完视频作为正样本。这一选择基于以下考量：可用的隐式用户历史数据在量级上远超显式反馈，使我们能够为长尾内容生成推荐，而在这些领域显式反馈极为稀疏。

高效的极端多分类 (*Efficient Extreme Multiclass*)

为了在数百万类别下高效训练此类模型，我们采用从背景分布中采样负类（“候选采样”）的技术，并通过重要性加权对采样偏差进行校正。对于每个样本，交叉熵损失在真实标签和采样的负类上同时被最小化。实践中采样数千个负样本，相比传统 softmax 实现了超过 100 倍的加速。层次化 softmax 是另一种常见替代方案，但我们未能达到可比的精度。在层次化 softmax 中，遍历树中每个节点需要在通常不相关的类别集合之间进行判别，这使分类问题更加困难，导致性能下降。

在服务阶段，需要计算最可能的  $N$  个类别（视频）以选出排名前  $N$  的视频呈现给用户。在数十毫秒的严格服务延迟下对数百万候选项打分，需要一种次线性复杂度的近似评分方案。YouTube 此前的系统依赖哈希方法，本文描述的分类器采用了类似的方案。由于服务时不需要 softmax 输出层的校准似然值，评分问题退化为点积空间中的最近邻搜索，可使用通用库实现。我们发现 A/B 实验结果对最近邻搜索算法的选择并不十分敏感。

### 8.4.2 模型架构

受连续词袋语言模型的启发，我们在固定词表中为每个视频学习高维嵌入，并将这些嵌入输入前馈神经网络。用户的观看历史由一个变长的稀疏视频 ID 序列表示，通过嵌入映射为稠密向量表示。网络需要固定大小的稠密输入，实验表明对嵌入取平均在多种策略（求和、逐元素最大值等）中表现最佳。重要的是，嵌入与其他所

有模型参数通过正常的梯度下降反向传播联合学习。特征被拼接成一个宽的第一层，随后是若干全连接整流线性单元（ReLU）层。图 3 展示了加入以下所述非视频观看特征后的通用网络架构。

### 8.4.3 异质信号

将深度神经网络作为矩阵分解推广的一个关键优势在于，任意连续型和类别型特征都可以方便地加入模型。搜索历史的处理方式与观看历史类似——每个查询被切分为 unigram 和 bigram，每个 token 被嵌入。取平均后，用户经过分词和嵌入处理的查询序列代表了一份压缩的稠密搜索历史。人口统计特征对于为新用户提供先验信息、使推荐行为合理化至关重要。用户的地理区域和设备被嵌入后拼接。性别、登录状态和年龄等简单二值型和连续型特征以归一化到  $[0, 1]$  的实数值直接输入网络。

“样本年龄”特征（“Example Age” Feature）

每秒都有大量视频上传至 YouTube。推荐新上传的（“新鲜”）内容对 YouTube 作为产品而言极为重要。我们持续观察到用户偏好新鲜内容，但不以牺牲相关性为代价。除了简单推荐用户想看的新视频这一一阶效应外，还存在引导和传播病毒式内容这一关键的二阶现象。

机器学习系统通常对历史存在隐式偏差，因为它们是从历史样本中预测未来行为训练而来的。视频流行度的分布高度非平稳，但我们推荐系统所产生的语料库上的多项分布将反映数周训练窗口内的平均观看可能性。为了纠正这一偏差，我们在训练时将训练样本的年龄（age）作为特征输入。在服务时，该特征被设为零（或略为负值），以体现模型是在训练窗口末端做出预测。

图 4 展示了该方法在一个任意选取的视频上的有效性。

### 8.4.4 标签与上下文选择

需要特别强调的是，推荐往往涉及求解一个代理问题（*surrogate problem*）并将结果迁移到特定场景。一个经典例子是“准确预测评分可以带来有效电影推荐”这一假设。我们发现，代理学习问题的选择对 A/B 测试中的性能影响极大，但在离线实验中却很难衡量。

训练样本来自所有 YouTube 观看记录（包括嵌入在其他网站上的观看），而非仅限于我们推荐产生的观看。否则，新内容将极难浮现，推荐系统也会对利用（*exploitation*）产生过度偏倚。若用户通过推荐以外的途径发现视频，我们希望能够通过协同过滤迅速将这一发现传播给其他用户。另一个改善在线指标的关键洞见是：为每位用户生成固定数量的训练样本，从而在损失函数中对用户进行等权重处理，防止少数高活跃用户主导损失。

有些违反直觉的是，为防止模型利用网站结构并对代理问题过拟合，必须谨慎地对分类器隐藏某些信息。举例来说，若用户刚刚搜索了“taylor swift”，由于问题被建模为预测下一个观看视频，给定该信息的分类器会预测最可能被观看的视频正是对应搜索结果页上出现的视频。不出所料，将用户最近的搜索页面作为首页推荐的表现极差。通过丢弃序列信息、以无序词袋表示搜索查询，分类器不再能直接感知标签的来源。

视频的自然消费模式通常导致非常不对称的共同观看概率。系列剧通常按顺序观看，用户往往从某类型中最广为人知的内容开始，逐渐深入到更细分的领域。因此，我们发现预测用户的下一次观看，比预测随机留出的观看效果更好（图 5）。许多协同过滤系统通过留出随机一项并从用户历史中的其他项进行预测来隐式选择标签和上下文（图 5a），这会泄露未来信息并忽略非对称的消费模式。与之相反，我们通过选择一次随机观看来“回滚”用户历史，仅将用户在留出标签观看之前的行为作为输入（图 5b）。

### 8.4.5 特征与深度实验

如图 6 所示，增加特征和深度能显著提升留出数据上的精确率。在这些实验中，100 万个视频词表和 100 万个搜索词令牌词表各以 256 个浮点数嵌入，最大词袋大小为 50 条最近观看和 50 条最近搜索。softmax 层在相同的 100 万视频类别上输出维度为 256 的多项分布（可视为一个独立的输出视频嵌入）。这些模型在所有 YouTube 用户上训练至收敛，对应在数据上进行若干个 epoch。网络结构遵循常见的“塔形”模式——网络底层最宽，每

层隐藏层的单元数减半（类似图 3）。深度为零的网络实际上是一个线性分解方案，与前代系统的表现非常相似。不断增加宽度和深度，直到增量收益减弱、收敛变得困难为止：

- 深度 0：一个线性层，将拼接层变换至 softmax 维度 256
- 深度 1：256 ReLU
- 深度 2：512 ReLU → 256 ReLU
- 深度 3：1024 ReLU → 512 ReLU → 256 ReLU
- 深度 4：2048 ReLU → 1024 ReLU → 512 ReLU → 256 ReLU

## 8.5 排序

排序的核心职责是利用曝光数据，针对特定用户界面对候选预测结果进行专项化校准。例如，某用户总体上以较高概率观看某视频，但由于缩略图的选择，未必会点击该视频在首页上的特定曝光位。在排序阶段，由于只需对数百个视频打分（而非候选生成阶段的数百万个），我们能够获取大量描述视频与用户关系的特征。排序对于融合来自不同候选来源、分数不可直接比较的结果同样至关重要。

我们使用与候选生成架构类似的深度神经网络，通过逻辑回归（图 7）为每个视频曝光独立打分，然后按分数排序返回给用户。最终排序目标依据在线 A/B 测试结果持续调整，但通常是每次曝光期望观看时长的简单函数。按点击率排序往往会推广用户不会看完的欺骗性视频（“标题党”），而观看时长能更好地衡量用户参与度。

### 8.5.1 特征表示

我们的特征遵循传统的类别型与连续型/有序型特征分类体系。所使用的类别型特征在基数上差异悬殊——有些是二值型（如用户是否登录），另一些则有数百万个可能取值（如用户的最近一次搜索词）。特征还按照是贡献单一值（“单值型”）还是一组值（“多值型”）进一步区分。单值类别型特征的典型例子是当前打分的视频 ID，对应的多值特征则可以是用户观看的最近  $N$  个视频 ID 的词袋。此外，特征还按是否描述候选项属性（“曝光”特征）或用户/上下文属性（“查询”特征）进行分类。查询特征每次请求只计算一次，而曝光特征则需对每个打分候选项分别计算。

#### 特征工程 (Feature Engineering)

我们的排序模型通常使用数百个特征，类别型与连续型特征大致各占一半。尽管深度学习有望减轻手工特征工程的负担，但我们的原始数据性质使其无法直接输入前馈神经网络。我们仍然投入大量工程资源将用户和视频数据转化为有用的特征，核心挑战在于如何表示用户行为的时序序列，以及这些行为与当前打分视频曝光之间的关系。

我们观察到最重要的信号，是描述用户与候选项本身及相似候选项的历史交互情况的特征，与其他排序广告经验相吻合。例如，考虑用户与当前打分视频所属频道的历史关系——用户观看了该频道多少视频？用户最近一次观看该主题内容是什么时候？这些描述用户在相关内容上历史行为的连续型特征尤为强大，因为它们能在不同内容之间良好泛化。我们还发现，将候选生成阶段的信息以特征形式传递给排序阶段至关重要，例如：哪些来源提名了这个候选视频？它们分别给出了怎样的分数？

描述历史视频曝光频率的特征对于在推荐中引入“扰动 (churn)”同样关键——确保连续请求不会返回完全相同的列表。若某视频最近被推荐给用户但未被观看，模型将在下次页面加载时自然地降低该曝光的排名。实时提供最新的曝光和观看历史本身就是一项重大的工程挑战，超出了本文讨论范围，但对于提供响应灵敏的推荐至关重要。

#### 嵌入类别型特征 (Embedding Categorical Features)

与候选生成类似，我们使用嵌入将稀疏类别型特征映射为适合神经网络的稠密表示。每个唯一 ID 空间（“词表”）有一个独立的可学习嵌入，其维度大致与唯一取值数量的对数成正比。这些词表是在训练开始前对数据进行一次遍历后构建的简单查找表。极高基数的 ID 空间（如视频 ID 或搜索词）通过仅保留按点击曝光频率排

序的前  $N$  个进行截断。词表外的取值简单地映射到零嵌入。与候选生成相同，多值类别型特征的嵌入在输入网络前取平均。

重要的是，同一 ID 空间内的类别型特征共享底层嵌入。例如，存在一个单一的全局视频 ID 嵌入，被许多不同特征使用（曝光的视频 ID、用户最近观看的视频 ID、“种子”推荐的视频 ID 等）。尽管共享嵌入，每个特征仍单独输入网络，使上层网络能够为每个特征学习专门的表示。共享嵌入有助于改善泛化、加速训练并降低内存需求。绝大多数模型参数都集中在这些高基数嵌入空间中——例如，100 万个 ID 在 32 维空间中嵌入的参数量是 2048 单元全连接层的 7 倍。

#### 连续型特征归一化 (Normalizing Continuous Features)

神经网络对输入的尺度和分布极为敏感，而决策树集成等方法则对单个特征的缩放不变。我们发现，对连续型特征进行合理归一化对收敛至关重要。对于分布为  $f$  的连续型特征  $x$ ，通过累积分布函数  $\tilde{x} = \int_{-\infty}^x df$  将其缩放至在  $[0, 1]$  上均匀分布，得到归一化特征  $\tilde{x}$ 。该积分在训练开始前对数据进行一次遍历，通过特征值分位数上的线性插值近似。

除原始归一化特征  $\tilde{x}$  外，我们还将其幂次  $\tilde{x}^2$  和  $\sqrt{\tilde{x}}$  一并输入，赋予网络更强的表达能力，使其能够轻松构建特征的超线性和次线性函数。实验表明，输入连续型特征的幂次能够改善离线准确率。

### 8.5.2 期望观看时长建模

我们的目标是根据正样本（视频曝光被点击）和负样本（曝光未被点击）的训练样本，预测期望观看时长。正样本被标注了用户观看该视频所花费的时间。为预测期望观看时长，我们采用加权逻辑回归技术，该技术正是为此目的而开发的。

模型在交叉熵损失下以逻辑回归训练（图 7）。然而，正样本（被点击的曝光）以该视频的实际观看时长加权，负样本（未被点击的曝光）则统一赋予单位权重。由此，逻辑回归所学习的赔率为  $\frac{\sum T_i}{N-k}$ ，其中  $N$  为训练样本总数， $k$  为正样本数， $T_i$  为第  $i$  个样本的观看时长。假设正样本比例较小（在我们的场景下确实如此），学习到的赔率近似为  $E[T](1+P)$ ，其中  $P$  为点击概率， $E[T]$  为该曝光的期望观看时长。由于  $P$  较小，该乘积接近于  $E[T]$ 。在推断时，我们使用指数函数  $e^x$  作为最终激活函数，以产生对期望观看时长的近似估计。

### 8.5.3 隐藏层实验

表 1 展示了我们在不同隐藏层配置下的次日留出数据结果。每种配置的指标值（“加权逐用户损失”）通过考察单次页面加载中呈现给用户的正样本（被点击）和负样本（未被点击）曝光对来计算。我们首先用模型对两个曝光打分：若负样本得分高于正样本，则认为正样本的观看时长被错误预测。加权逐用户损失即为留出曝光对中，被错误预测的观看时长占总观看时长的比例。

结果表明，增加隐藏层宽度和深度均能改善性能。然而，代价是推断所需的服务端 CPU 时间。1024 → 512 → 256 的 ReLU 配置在满足服务 CPU 预算的前提下取得了最佳结果。

对于 1024 → 512 → 256 模型，我们尝试仅输入归一化连续型特征而不加入其幂次，损失增加了 0.2%。在相同隐藏层配置下，我们还训练了一个对正负样本等权重处理的模型，正如预期，观看时长加权损失显著上升了 4.1%。

## 8.6 结论

我们描述了用于 YouTube 视频推荐的深度神经网络架构，将其拆分为两个独立问题：候选生成与排序。

我们的深度协同过滤模型能够有效融合多种信号，并通过多层深度对信号间的交互建模，优于 YouTube 此前使用的矩阵分解方法。在为推荐选择代理问题时，艺术性多于科学性——我们发现，预测用户的未来观看在线指标上表现优异，这得益于它能捕捉非对称的共同观看行为，并防止未来信息的泄露。对分类器隐藏判别性信号同样是取得良好结果的关键——否则模型会对代理问题过拟合，无法良好迁移到首页场景。

我们证明，将训练样本的年龄作为输入特征，能够消除模型对历史的隐式偏倚，使模型能够表示视频流行度的时间依赖行为。这改善了离线留出精确率，并在 A/B 测试中显著提升了近期上传视频的观看时长。

排序是一个更为经典的机器学习问题，但我们的深度学习方法在观看时长预测上优于此前的线性和基于树的方法。推荐系统尤其受益于描述用户对候选项历史行为的专项特征。深度神经网络要求对类别型和连续型特征进行特殊表示，我们分别通过嵌入和分位数归一化进行处理。多层深度被证明能够有效建模数百个特征间的非线性交互。

逻辑回归通过对正样本以观看时长加权、对负样本赋予单位权重进行改进，使我们能够学习到对期望观看时长精确建模的赔率。该方法在以观看时长加权的排序评估指标上，远优于直接预测点击率的方法。

# 第 9 章 Deep Neural Networks for YouTube Recommendations

## 9.1 Abstract

YouTube represents one of the largest scale and most sophisticated industrial recommendation systems in existence. In this paper, we describe the system at a high level and focus on the dramatic performance improvements brought by deep learning. The paper is split according to the classic two-stage information retrieval dichotomy: first, we detail a deep candidate generation model and then describe a separate deep ranking model. We also provide practical lessons and insights derived from designing, iterating and maintaining a massive recommendation system with enormous user-facing impact.

**Keywords:** recommender system; deep learning; scalability

## 9.2 Introduction

YouTube is the world's largest platform for creating, sharing and discovering video content. YouTube recommendations are responsible for helping more than a billion users discover personalized content from an ever-growing corpus of videos. In this paper we will focus on the immense impact deep learning has recently had on the YouTube video recommendations system. Figure 1 illustrates the recommendations on the YouTube mobile app home.

Recommending YouTube videos is extremely challenging from three major perspectives:

- **Scale:** Many existing recommendation algorithms proven to work well on small problems fail to operate on our scale. Highly specialized distributed learning algorithms and efficient serving systems are essential for handling YouTube's massive user base and corpus.
- **Freshness:** YouTube has a very dynamic corpus with many hours of video are uploaded per second. The recommendation system should be responsive enough to model newly uploaded content as well as the latest actions taken by the user. Balancing new content with well-established videos can be understood from an exploration/exploitation perspective.
- **Noise:** Historical user behavior on YouTube is inherently difficult to predict due to sparsity and a variety of unobservable external factors. We rarely obtain the ground truth of user satisfaction and instead model noisy implicit feedback signals. Furthermore, metadata associated with content is poorly structured without a well defined ontology. Our algorithms need to be robust to these particular characteristics of our training data.

In conjugation with other product areas across Google, YouTube has undergone a fundamental paradigm shift towards using deep learning as a general-purpose solution for nearly all learning problems. Our system is built on Google Brain [dean2012large] which was recently open sourced as TensorFlow [tensorflow2015]. TensorFlow provides a flexible framework for experimenting with various deep neural network architectures using large-scale distributed training. Our models learn approximately one billion parameters and are trained on hundreds of billions of examples.

In contrast to vast amount of research in matrix factorization methods [su2009survey], there is relatively little work using deep neural networks for recommendation systems. Neural networks are used for recommending news in [oh2014personalized], citations in [huang2015neural] and review ratings in [tang2015user]. Collaborative filtering is formulated as a deep neural network in [wang2015collaborative] and autoencoders in [sedhain2015autorec]. Elkahky et al. used deep learning for cross domain user modeling [elkahky2015multi]. In a content-based setting, Burges et al. used deep neural networks for music recommendation [vandennoord2013deep].

The paper is organized as follows: A brief system overview is presented in Section 2. Section 3 describes the candidate generation model in more detail, including how it is trained and used to serve recommendations. Experimental results will

show how the model benefits from deep layers of hidden units and additional heterogeneous signals. Section 4 details the ranking model, including how classic logistic regression is modified to train a model predicting expected watch time (rather than click probability). Experimental results will show that hidden layer depth is helpful as well in this situation. Finally, Section 5 presents our conclusions and lessons learned.

## 9.3 System Overview

The overall structure of our recommendation system is illustrated in Figure 2. The system is comprised of two neural networks: one for *candidate generation* and one for *ranking*.

The candidate generation network takes events from the user’s YouTube activity history as input and retrieves a small subset (hundreds) of videos from a large corpus. These candidates are intended to be generally relevant to the user with high precision. The candidate generation network only provides broad personalization via collaborative filtering. The similarity between users is expressed in terms of coarse features such as IDs of video watches, search query tokens and demographics.

Presenting a few “best” recommendations in a list requires a fine-level representation to distinguish relative importance among candidates with high recall. The ranking network accomplishes this task by assigning a score to each video according to a desired objective function using a rich set of features describing the video and user. The highest scoring videos are presented to the user, ranked by their score.

The two-stage approach to recommendation allows us to make recommendations from a very large corpus (millions) of videos while still being certain that the small number of videos appearing on the device are personalized and engaging for the user. Furthermore, this design enables blending candidates generated by other sources, such as those described in an earlier work [davidson2010youtube].

During development, we make extensive use of offline metrics (precision, recall, ranking loss, etc.) to guide iterative improvements to our system. However for the final determination of the effectiveness of an algorithm or model, we rely on A/B testing via live experiments. In a live experiment, we can measure subtle changes in click-through rate, watch time, and many other metrics that measure user engagement. This is important because live A/B results are not always correlated with offline experiments.

## 9.4 Candidate Generation

During candidate generation, the enormous YouTube corpus is winnowed down to hundreds of videos that may be relevant to the user. The predecessor to the recommender described here was a matrix factorization approach trained under rank loss [weston2011wsabie]. Early iterations of our neural network model mimicked this factorization behavior with shallow networks that only embedded the user’s previous watches. From this perspective, our approach can be viewed as a non-linear generalization of factorization techniques.

### 9.4.1 Recommendation as Classification

We pose recommendation as extreme multiclass classification where the prediction problem becomes accurately classifying a specific video watch  $w_t$  at time  $t$  among millions of videos  $i$  (classes) from a corpus  $V$  based on a user  $U$  and context  $C$ ,

$$P(w_t = i \mid U, C) = \frac{e^{v_i u}}{\sum_{j \in V} e^{v_j u}}$$

where  $u \in \mathbb{R}^N$  represents a high-dimensional “embedding” of the user, context pair and the  $v_j \in \mathbb{R}^N$  represent embeddings of each candidate video. In this setting, an embedding is simply a mapping of sparse entities (individual videos, users etc.)

into a dense vector in  $\mathbb{R}^N$ . The task of the deep neural network is to learn user embeddings  $u$  as a function of the user’s history and context that are useful for discriminating among videos with a softmax classifier.

Although explicit feedback mechanisms exist on YouTube (thumbs up/down, in-product surveys, etc.) we use the implicit feedback [oard1998implicit] of watches to train the model, where a user completing a video is a positive example. This choice is based on the orders of magnitude more implicit user history available, allowing us to produce recommendations deep in the tail where explicit feedback is extremely sparse.

#### *Efficient Extreme Multiclass*

To efficiently train such a model with millions of classes, we rely on a technique to sample negative classes from the background distribution (“candidate sampling”) and then correct for this sampling via importance weighting [jean2014using]. For each example the cross-entropy loss is minimized for the true label and the sampled negative classes. In practice several thousand negatives are sampled, corresponding to more than 100 times speedup over traditional softmax. A popular alternative approach is hierarchical softmax [morin2005hierarchical], but we weren’t able to achieve comparable accuracy. In hierarchical softmax, traversing each node in the tree involves discriminating between sets of classes that are often unrelated, making the classification problem much more difficult and degrading performance.

At serving time we need to compute the most likely  $N$  classes (videos) in order to choose the top  $N$  to present to the user. Scoring millions of items under a strict serving latency of tens of milliseconds requires an approximate scoring scheme sublinear in the number of classes. Previous systems at YouTube relied on hashing [weston2013label] and the classifier described here uses a similar approach. Since calibrated likelihoods from the softmax output layer are not needed at serving time, the scoring problem reduces to a nearest neighbor search in the dot product space for which general purpose libraries can be used [liu2004investigation]. We found that A/B results were not particularly sensitive to the choice of nearest neighbor search algorithm.

### 9.4.2 Model Architecture

Inspired by continuous bag of words language models [mikolov2013distributed], we learn high dimensional embeddings for each video in a fixed vocabulary and feed these embeddings into a feedforward neural network. A user’s watch history is represented by a variable-length sequence of sparse video IDs which is mapped to a dense vector representation via the embeddings. The network requires fixed-sized dense inputs and simply averaging the embeddings performed best among several strategies (sum, component-wise max, etc.). Importantly, the embeddings are learned jointly with all other model parameters through normal gradient descent backpropagation updates. Features are concatenated into a wide first layer, followed by several layers of fully connected Rectified Linear Units (ReLU) [glorot2011deep]. Figure 3 shows the general network architecture with additional non-video watch features described below.

### 9.4.3 Heterogeneous Signals

A key advantage of using deep neural networks as a generalization of matrix factorization is that arbitrary continuous and categorical features can be easily added to the model. Search history is treated similarly to watch history — each query is tokenized into unigrams and bigrams and each token is embedded. Once averaged, the user’s tokenized, embedded queries represent a summarized dense search history. Demographic features are important for providing priors so that the recommendations behave reasonably for new users. The user’s geographic region and device are embedded and concatenated. Simple binary and continuous features such as the user’s gender, logged-in state and age are input directly into the network as real values normalized to  $[0, 1]$ .

#### *“Example Age” Feature*

Many hours worth of videos are uploaded each second to YouTube. Recommending this recently uploaded (“fresh”) content is extremely important for YouTube as a product. We consistently observe that users prefer fresh content, though

not at the expense of relevance. In addition to the first-order effect of simply recommending new videos that users want to watch, there is a critical secondary phenomenon of bootstrapping and propagating viral content [jiang2014viral].

Machine learning systems often exhibit an implicit bias towards the past because they are trained to predict future behavior from historical examples. The distribution of video popularity is highly non-stationary but the multinomial distribution over the corpus produced by our recommender will reflect the average watch likelihood in the training window of several weeks. To correct for this, we feed the age of the training example as a feature during training. At serving time, this feature is set to zero (or slightly negative) to reflect that the model is making predictions at the very end of the training window.

Figure 4 demonstrates the efficacy of this approach on an arbitrarily chosen video [zayn2016pillowtalk].

#### 9.4.4 Label and Context Selection

It is important to emphasize that recommendation often involves solving a *surrogate problem* and transferring the result to a particular context. A classic example is the assumption that accurately predicting ratings leads to effective movie recommendations [amatriain2012building]. We have found that the choice of this surrogate learning problem has an outsized importance on performance in A/B testing but is very difficult to measure with offline experiments.

Training examples are generated from all YouTube watches (even those embedded on other sites) rather than just watches on the recommendations we produce. Otherwise, it would be very difficult for new content to surface and the recommender would be overly biased towards exploitation. If users are discovering videos through means other than our recommendations, we want to be able to quickly propagate this discovery to others via collaborative filtering. Another key insight that improved live metrics was to generate a fixed number of training examples per user, effectively weighting our users equally in the loss function. This prevented a small cohort of highly active users from dominating the loss.

Somewhat counter-intuitively, great care must be taken to *withhold information from the classifier* in order to prevent the model from exploiting the structure of the site and overfitting the surrogate problem. Consider as an example a case in which the user has just issued a search query for “taylor swift”. Since our problem is posed as predicting the next watched video, a classifier given this information will predict that the most likely videos to be watched are those which appear on the corresponding search results page for “taylor swift”. Unsurprisingly, reproducing the user’s last search page as homepage recommendations performs very poorly. By discarding sequence information and representing search queries with an unordered bag of tokens, the classifier is no longer directly aware of the origin of the label.

Natural consumption patterns of videos typically lead to very asymmetric co-watch probabilities. Episodic series are usually watched sequentially and users often discover artists in a genre beginning with the most broadly popular before focusing on smaller niches. We therefore found much better performance predicting the user’s next watch, rather than predicting a randomly held-out watch (Figure 5). Many collaborative filtering systems implicitly choose the labels and context by holding out a random item and predicting it from other items in the user’s history (5a). This leaks future information and ignores any asymmetric consumption patterns. In contrast, we “rollback” a user’s history by choosing a random watch and only input actions the user took before the held-out label watch (5b).

#### 9.4.5 Experiments with Features and Depth

Adding features and depth significantly improves precision on holdout data as shown in Figure 6. In these experiments, a vocabulary of 1M videos and 1M search tokens were embedded with 256 floats each in a maximum bag size of 50 recent watches and 50 recent searches. The softmax layer outputs a multinomial distribution over the same 1M video classes with a dimension of 256 (which can be thought of as a separate output video embedding). These models were trained until convergence over all YouTube users, corresponding to several epochs over the data. Network structure followed a common “tower” pattern in which the bottom of the network is widest and each successive hidden layer halves the number of units (similar to Figure 3). The depth zero network is effectively a linear factorization scheme which performed very similarly

to the predecessor system. Width and depth were added until the incremental benefit diminished and convergence became difficult:

- Depth 0: A linear layer simply transforms the concatenation layer to match the softmax dimension of 256
- Depth 1: 256 ReLU
- Depth 2: 512 ReLU  $\rightarrow$  256 ReLU
- Depth 3: 1024 ReLU  $\rightarrow$  512 ReLU  $\rightarrow$  256 ReLU
- Depth 4: 2048 ReLU  $\rightarrow$  1024 ReLU  $\rightarrow$  512 ReLU  $\rightarrow$  256 ReLU

## 9.5 Ranking

The primary role of ranking is to use impression data to specialize and calibrate candidate predictions for the particular user interface. For example, a user may watch a given video with high probability generally but is unlikely to click on the specific homepage impression due to the choice of thumbnail image. During ranking, we have access to many more features describing the video and the user’s relationship to the video because only a few hundred videos are being scored rather than the millions scored in candidate generation. Ranking is also crucial for ensembling different candidate sources whose scores are not directly comparable.

We use a deep neural network with similar architecture as candidate generation to assign an independent score to each video impression using logistic regression (Figure 7). The list of videos is then sorted by this score and returned to the user. Our final ranking objective is constantly being tuned based on live A/B testing results but is generally a simple function of expected watch time per impression. Ranking by click-through rate often promotes deceptive videos that the user does not complete (“clickbait”) whereas watch time better captures engagement [meyerson2012watchtime, yi2014beyond].

### 9.5.1 Feature Representation

Our features are segregated with the traditional taxonomy of categorical and continuous/ordinal features. The categorical features we use vary widely in their cardinality — some are binary (e.g. whether the user is logged-in) while others have millions of possible values (e.g. the user’s last search query). Features are further split according to whether they contribute only a single value (“univalent”) or a set of values (“multivalent”). An example of a univalent categorical feature is the video ID of the impression being scored, while a corresponding multivalent feature might be a bag of the last  $N$  video IDs the user has watched. We also classify features according to whether they describe properties of the item (“impression”) or properties of the user/context (“query”). Query features are computed once per request while impression features are computed for each item scored.

#### *Feature Engineering*

We typically use hundreds of features in our ranking models, roughly split evenly between categorical and continuous. Despite the promise of deep learning to alleviate the burden of engineering features by hand, the nature of our raw data does not easily lend itself to be input directly into feedforward neural networks. We still expend considerable engineering resources transforming user and video data into useful features. The main challenge is in representing a temporal sequence of user actions and how these actions relate to the video impression being scored.

We observe that the most important signals are those that describe a user’s previous interaction with the item itself and other similar items, matching others’ experience in ranking ads [he2014practical]. As an example, consider the user’s past history with the channel that uploaded the video being scored — how many videos has the user watched from this channel? When was the last time the user watched a video on this topic? These continuous features describing past user actions on related items are particularly powerful because they generalize well across disparate items. We have also found it crucial to propagate information from candidate generation into ranking in the form of features, e.g. which sources nominated this video candidate? What scores did they assign?

Features describing the frequency of past video impressions are also critical for introducing “churn” in recommendations (successive requests do not return identical lists). If a user was recently recommended a video but did not watch it then the model will naturally demote this impression on the next page load. Serving up-to-the-second impression and watch history is an engineering feat onto itself outside the scope of this paper, but is vital for producing responsive recommendations.

#### *Embedding Categorical Features*

Similar to candidate generation, we use embeddings to map sparse categorical features to dense representations suitable for neural networks. Each unique ID space (“vocabulary”) has a separate learned embedding with dimension that increases approximately proportional to the logarithm of the number of unique values. These vocabularies are simple look-up tables built by passing over the data once before training. Very large cardinality ID spaces (e.g. video IDs or search query terms) are truncated by including only the top  $N$  after sorting based on their frequency in clicked impressions. Out-of-vocabulary values are simply mapped to the zero embedding. As in candidate generation, multivalent categorical feature embeddings are averaged before being fed in to the network.

Importantly, categorical features in the same ID space also share underlying embeddings. For example, there exists a single global embedding of video IDs that many distinct features use (video ID of the impression, last video ID watched by the user, video ID that “seeded” the recommendation, etc.). Despite the shared embedding, each feature is fed separately into the network so that the layers above can learn specialized representations per feature. Sharing embeddings is important for improving generalization, speeding up training and reducing memory requirements. The overwhelming majority of model parameters are in these high-cardinality embedding spaces — for example, one million IDs embedded in a 32 dimensional space have 7 times more parameters than fully connected layers 2048 units wide.

#### *Normalizing Continuous Features*

Neural networks are notoriously sensitive to the scaling and distribution of their inputs [ioffe2015batch] whereas alternative approaches such as ensembles of decision trees are invariant to scaling of individual features. We found that proper normalization of continuous features was critical for convergence. A continuous feature  $x$  with distribution  $f$  is transformed to  $\tilde{x}$  by scaling the values such that the feature is equally distributed in  $[0, 1)$  using the cumulative distribution,  $\tilde{x} = \int_{-\infty}^x df$ . This integral is approximated with linear interpolation on the quantiles of the feature values computed in a single pass over the data before training begins.

In addition to the raw normalized feature  $\tilde{x}$ , we also input powers  $\tilde{x}^2$  and  $\sqrt{\tilde{x}}$ , giving the network more expressive power by allowing it to easily form super- and sub-linear functions of the feature. Feeding powers of continuous features was found to improve offline accuracy.

### 9.5.2 Modeling Expected Watch Time

Our goal is to predict expected watch time given training examples that are either positive (the video impression was clicked) or negative (the impression was not clicked). Positive examples are annotated with the amount of time the user spent watching the video. To predict expected watch time we use the technique of weighted logistic regression, which was developed for this purpose.

The model is trained with logistic regression under cross-entropy loss (Figure 7). However, the positive (clicked) impressions are weighted by the observed watch time on the video. Negative (unclicked) impressions all receive unit weight. In this way, the odds learned by the logistic regression are  $\frac{\sum T_i}{N-k}$  where  $N$  is the number of training examples,  $k$  is the number of positive impressions, and  $T_i$  is the watch time of the  $i$ th impression. Assuming the fraction of positive impressions is small (which is true in our case), the learned odds are approximately  $E[T](1 + P)$ , where  $P$  is the click probability and  $E[T]$  is the expected watch time of the impression. Since  $P$  is small, this product is close to  $E[T]$ . For inference we use the exponential function  $e^x$  as the final activation function to produce these odds that closely estimate expected watch time.

### 9.5.3 Experiments with Hidden Layers

Table 1 shows the results we obtained on next-day holdout data with different hidden layer configurations. The value shown for each configuration (“weighted, per-user loss”) was obtained by considering both positive (clicked) and negative (unclicked) impressions shown to a user on a single page. We first score these two impressions with our model. If the negative impression receives a higher score than the positive impression, then we consider the positive impression’s watch time to be *mispredicted watch time*. Weighted, per-user loss is then the total amount mispredicted watch time as a fraction of total watch time over heldout impression pairs.

These results show that increasing the width of hidden layers improves results, as does increasing their depth. The trade-off, however, is server CPU time needed for inference. The configuration of a 1024-wide ReLU followed by a 512-wide ReLU followed by a 256-wide ReLU gave us the best results while enabling us to stay within our serving CPU budget.

For the 1024  $\rightarrow$  512  $\rightarrow$  256 model we tried only feeding the normalized continuous features without their powers, which increased loss by 0.2%. With the same hidden layer configuration, we also trained a model where positive and negative examples are weighted equally. Unsurprisingly, this increased the watch time-weighted loss by a dramatic 4.1%.

## 9.6 Conclusions

We have described our deep neural network architecture for recommending YouTube videos, split into two distinct problems: candidate generation and ranking.

Our deep collaborative filtering model is able to effectively assimilate many signals and model their interaction with layers of depth, outperforming previous matrix factorization approaches used at YouTube [weston2011wsabie]. There is more art than science in selecting the surrogate problem for recommendations and we found classifying a future watch to perform well on live metrics by capturing asymmetric co-watch behavior and preventing leakage of future information. Withholding discriminative signals from the classifier was also essential to achieving good results — otherwise the model would overfit the surrogate problem and not transfer well to the homepage.

We demonstrated that using the age of the training example as an input feature removes an inherent bias towards the past and allows the model to represent the time-dependent behavior of popular of videos. This improved offline holdout precision results and increased the watch time dramatically on recently uploaded videos in A/B testing.

Ranking is a more classical machine learning problem yet our deep learning approach outperformed previous linear and tree-based methods for watch time prediction. Recommendation systems in particular benefit from specialized features describing past user behavior with items. Deep neural networks require special representations of categorical and continuous features which we transform with embeddings and quantile normalization, respectively. Layers of depth were shown to effectively model non-linear interactions between hundreds of features.

Logistic regression was modified by weighting training examples with watch time for positive examples and unity for negative examples, allowing us to learn odds that closely model expected watch time. This approach performed much better on watch-time weighted ranking evaluation metrics compared to predicting click-through rate directly.